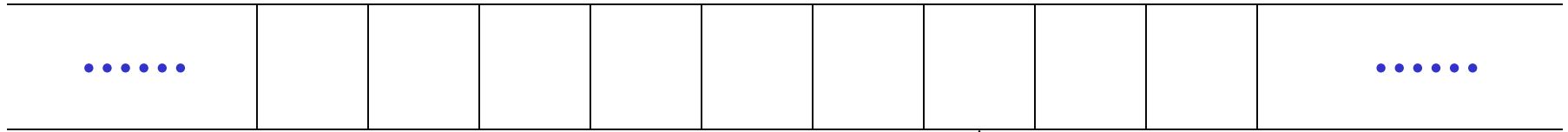


Turing Machines

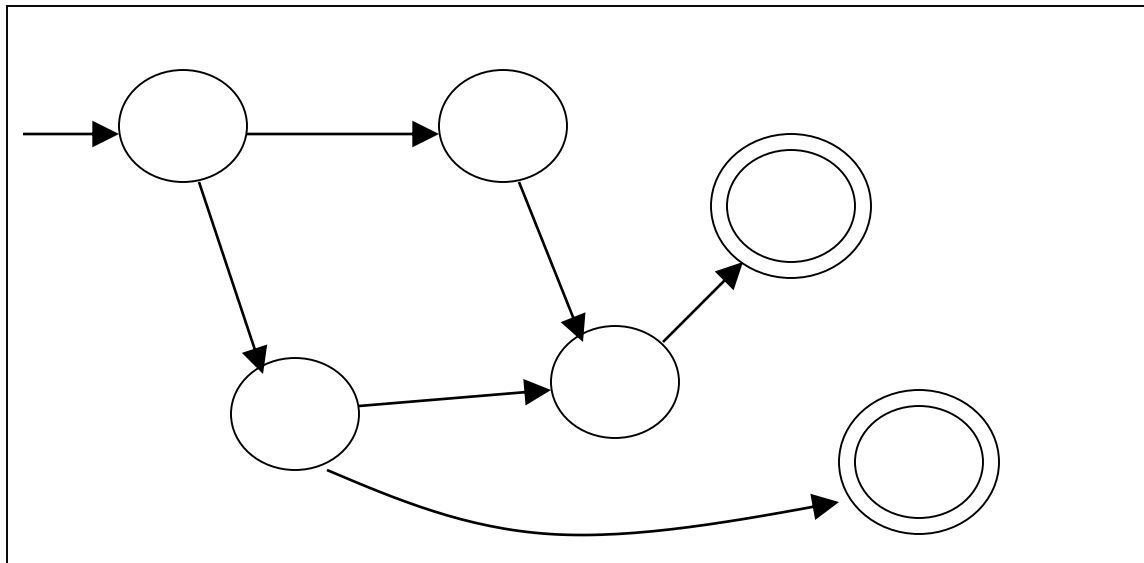
A Turing Machine

Tape



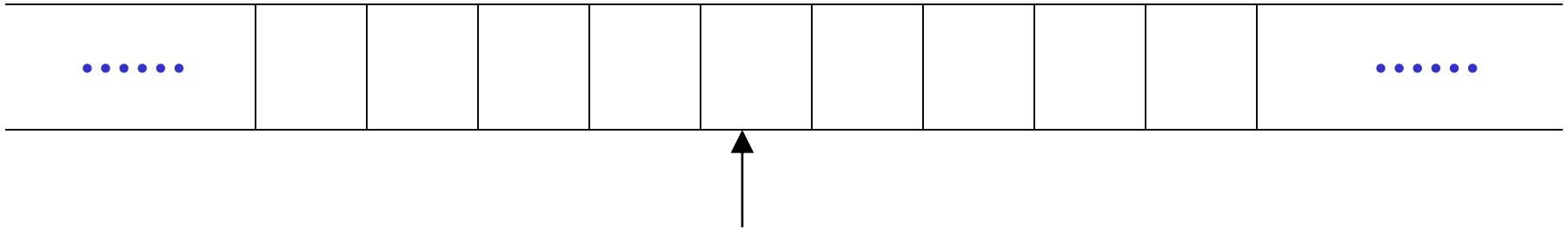
Read-Write head

Control Unit



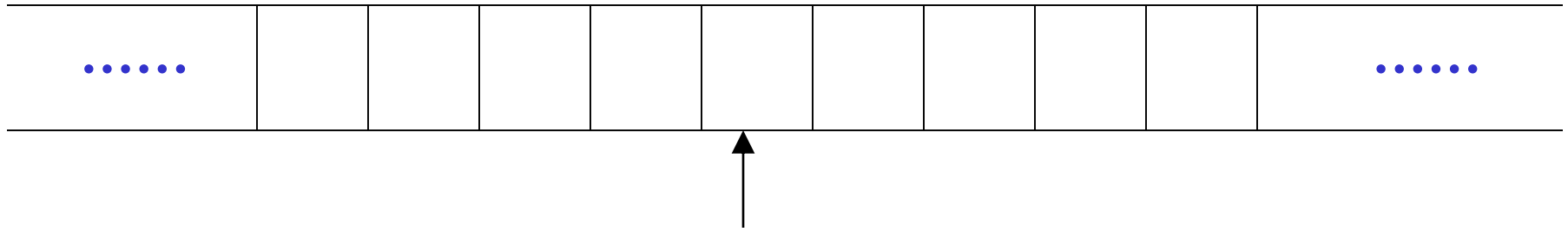
The Tape

No boundaries -- infinite length



Read-Write head

The head moves Left or Right



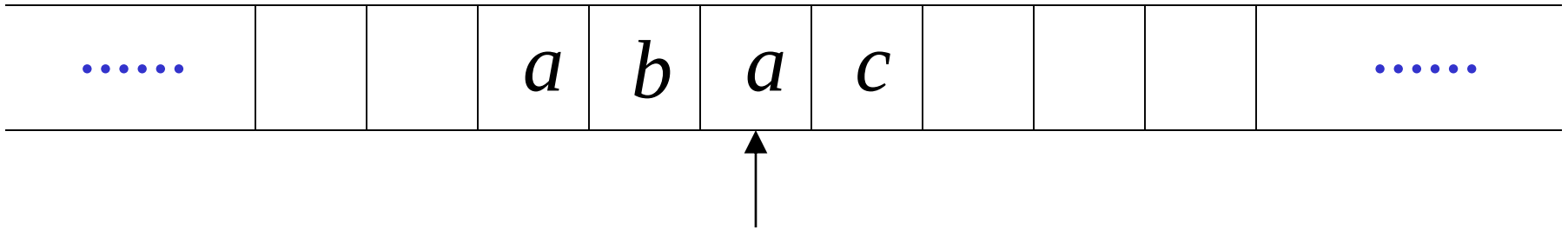
Read-Write head

The head at each transition (time step):

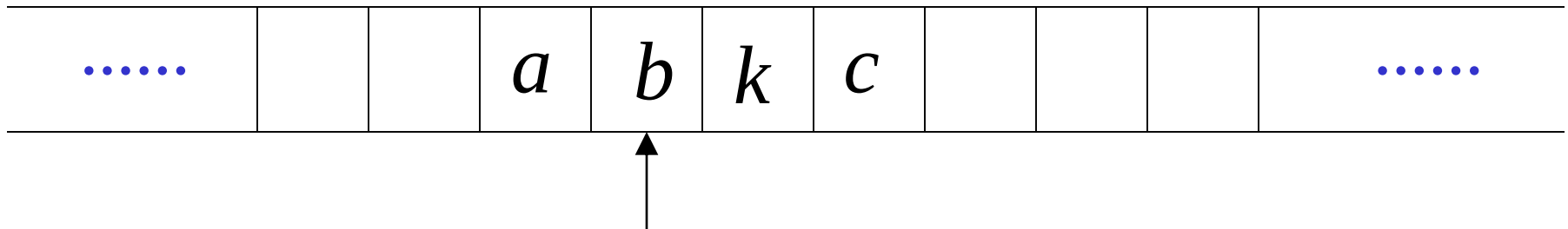
1. Reads a symbol
2. Writes a symbol
3. Moves Left or Right

Example:

Time 0



Time 1

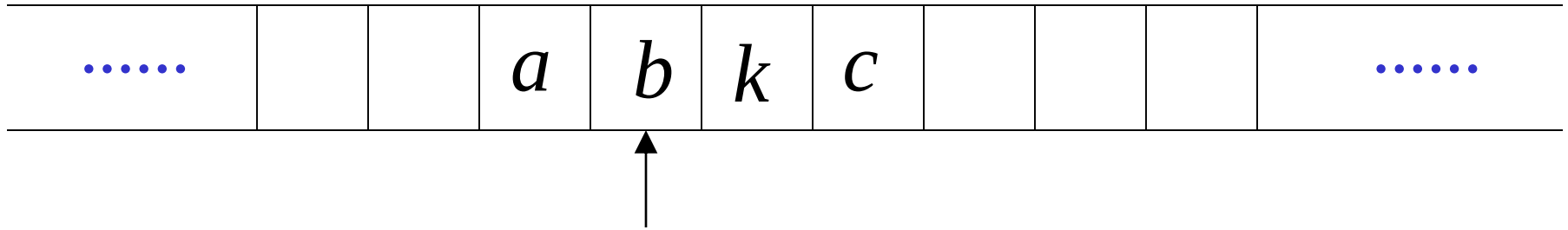


1. Reads *a*

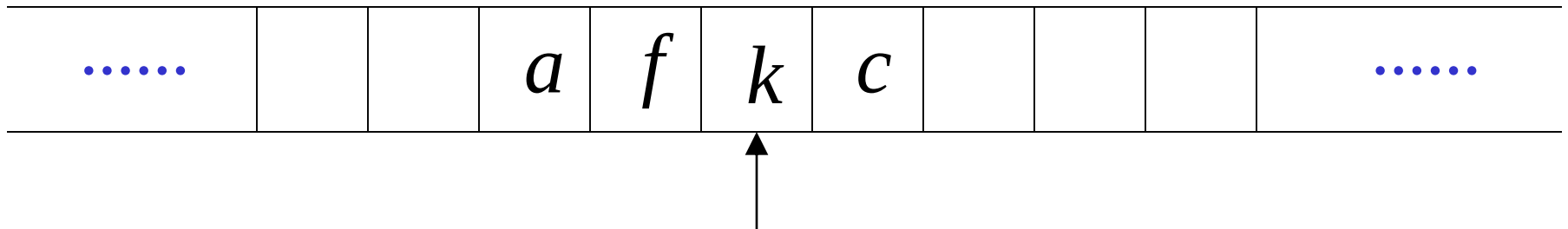
2. Writes *k*

3. Moves Left

Time 1

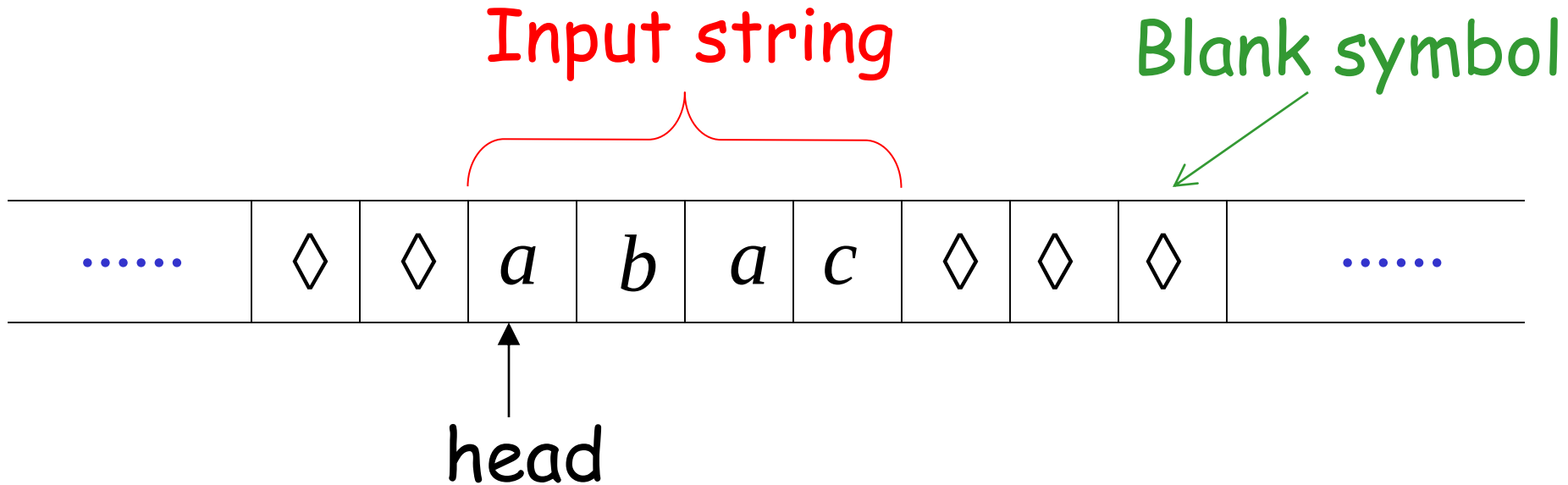


Time 2



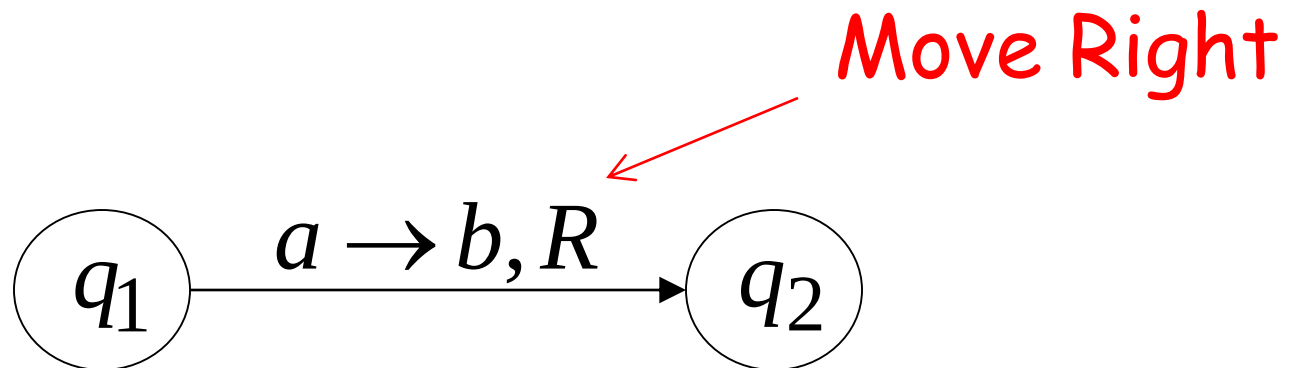
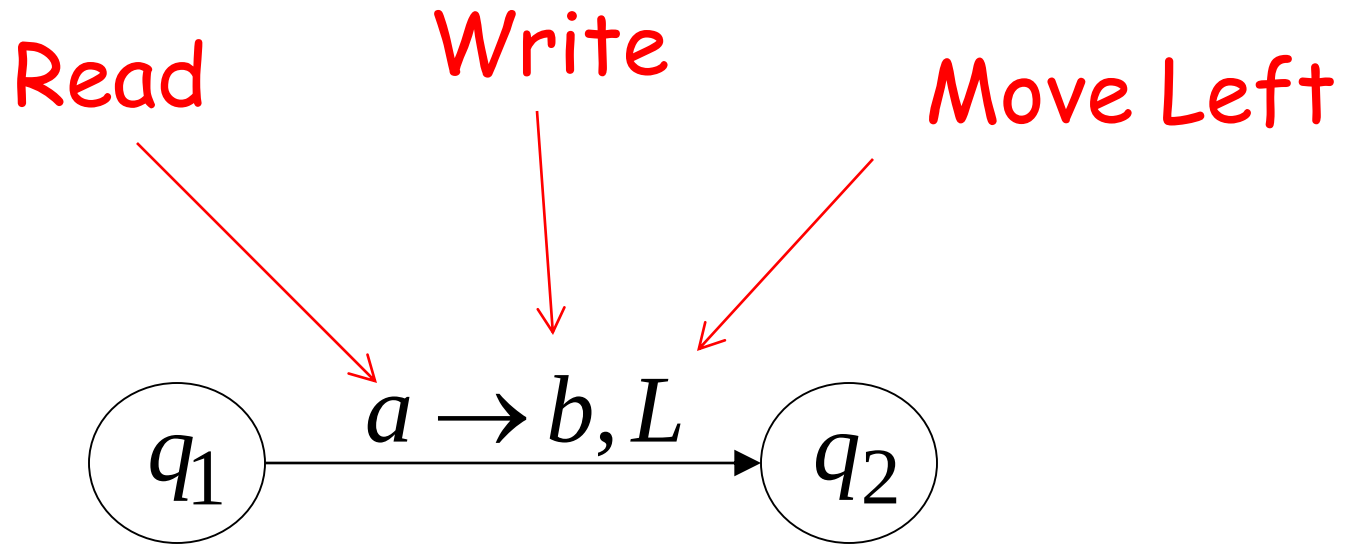
1. Reads *b*
2. Writes *f*
3. Moves Right

The Input String



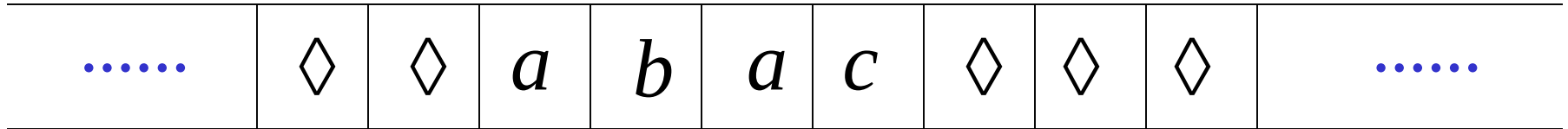
Head starts at the leftmost position of the input string

States & Transitions

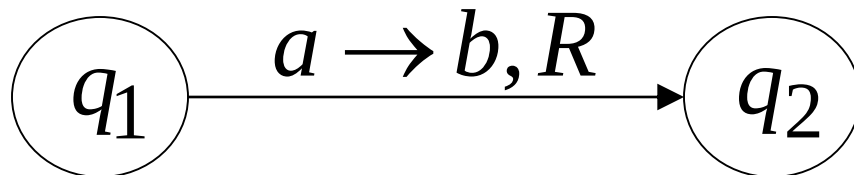


Example:

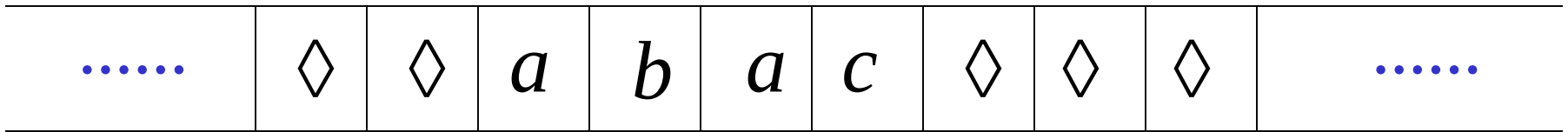
Time 1



current state

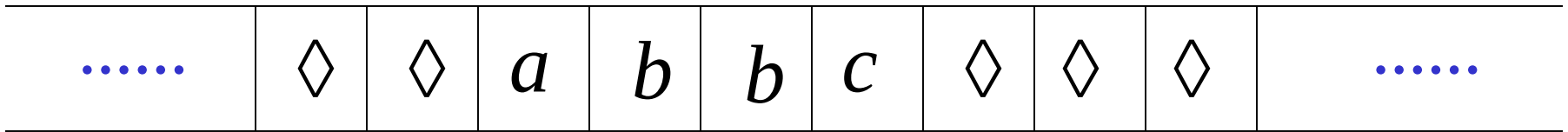


Time 1

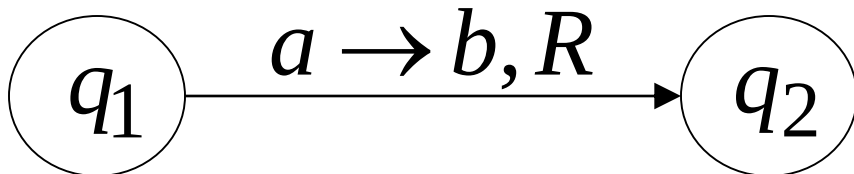


q_1

Time 2

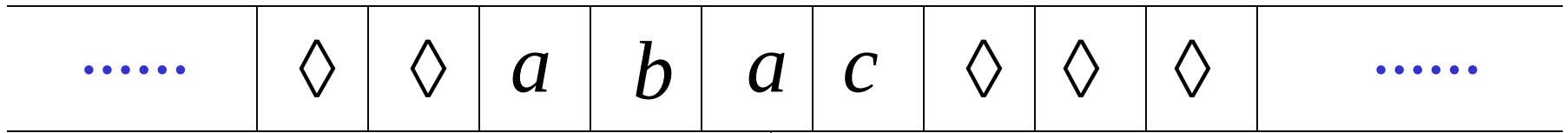


q_2



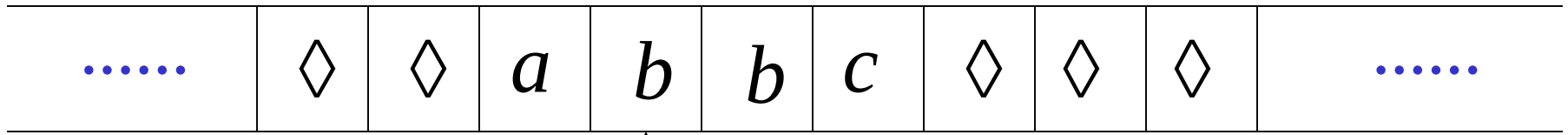
Example:

Time 1

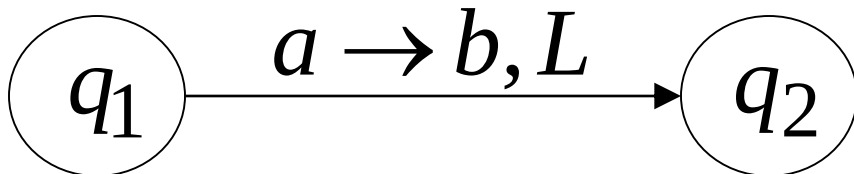


q_1

Time 2

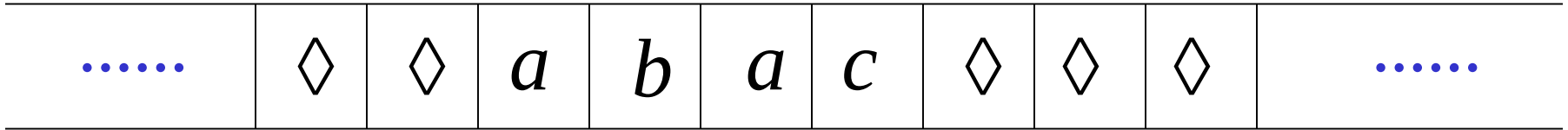


q_2



Example:

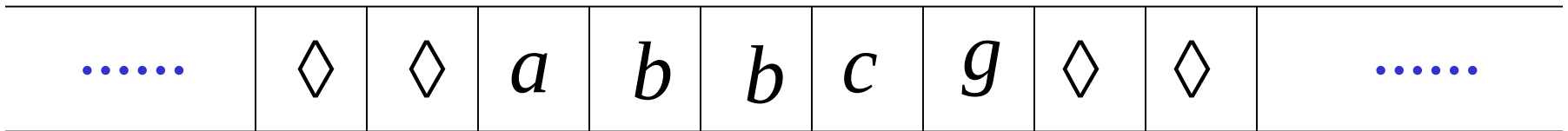
Time 1



q_1

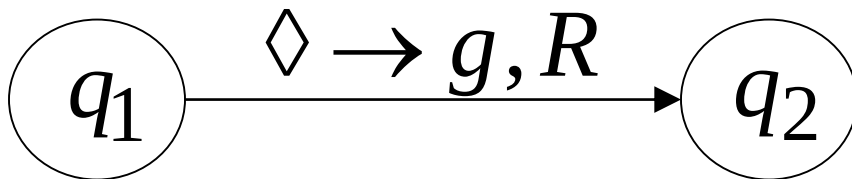
An upward-pointing arrow from the label q_1 to the diamond symbol in the 8th cell of the Time 1 memory layout.

Time 2



q_2

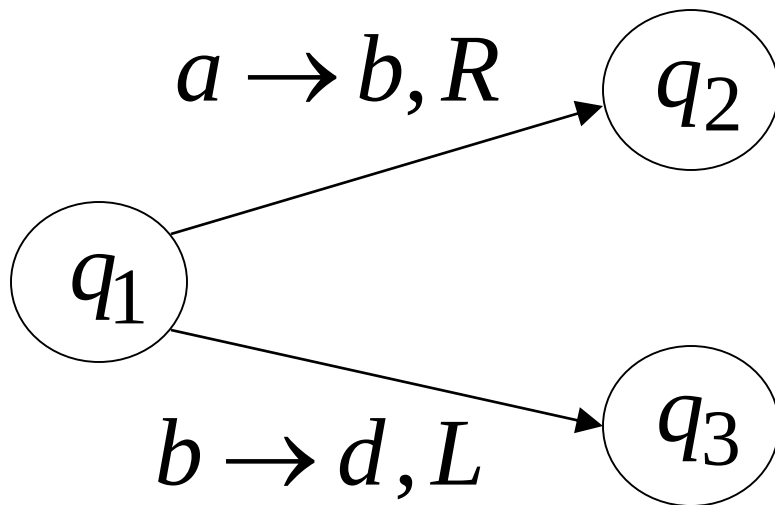
An upward-pointing arrow from the label q_2 to the diamond symbol in the 9th cell of the Time 2 memory layout.



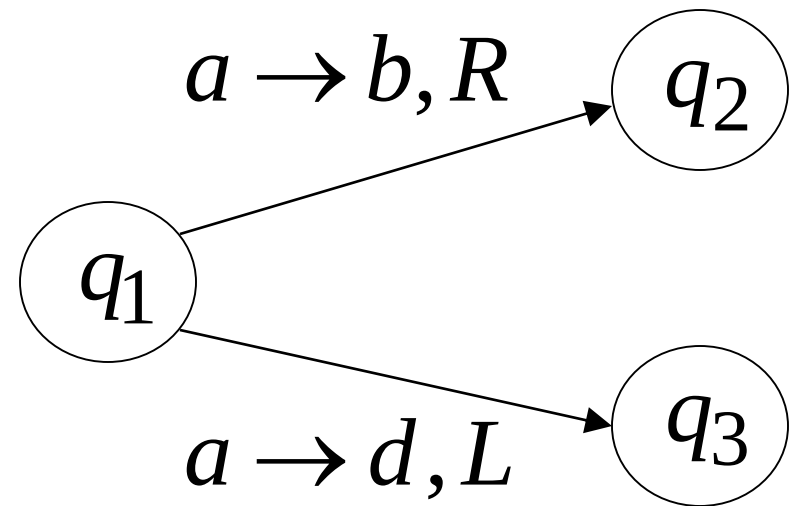
Determinism

Turing Machines are deterministic

Allowed

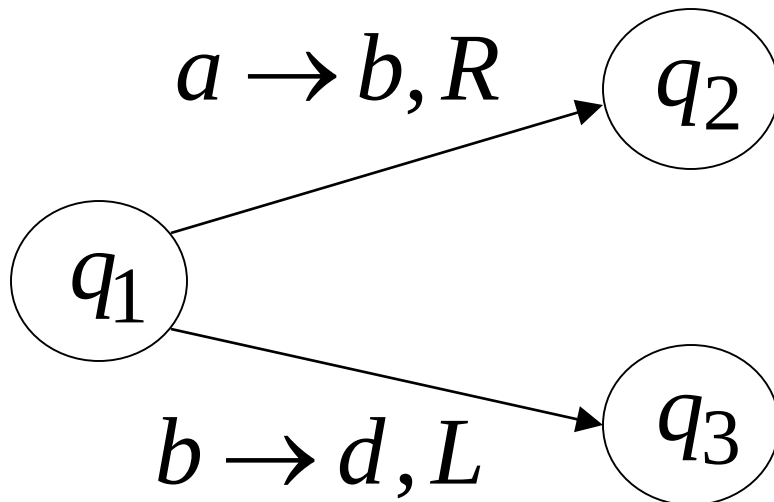
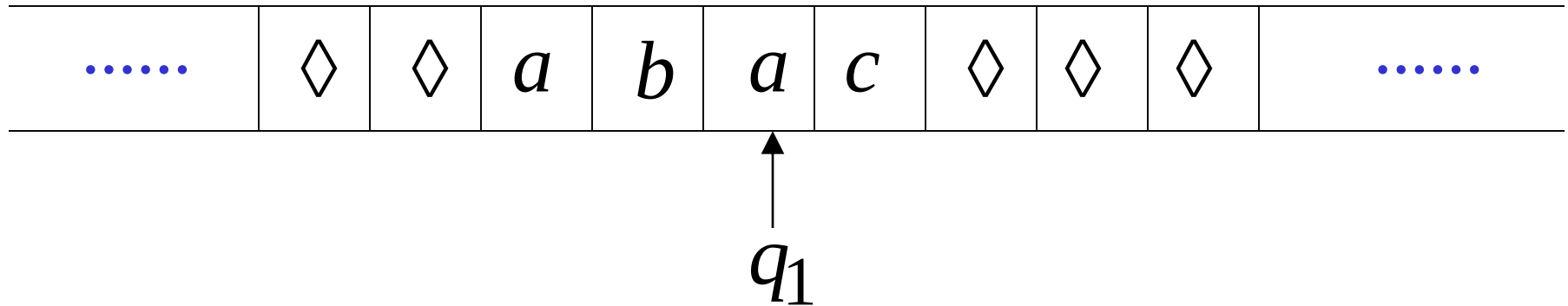


Not Allowed



Partial Transition Function

Example:



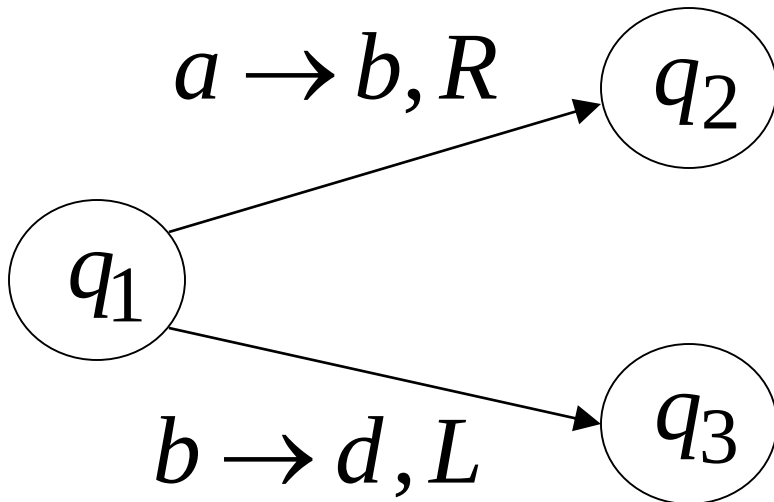
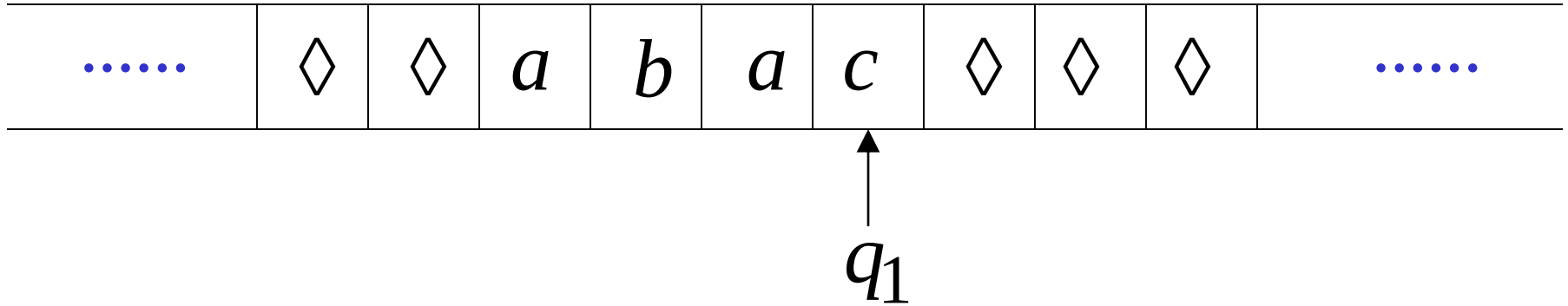
Allowed:

No transition
for input symbol c

Halting

The machine *halts* if there are
no possible transitions to follow

Example:



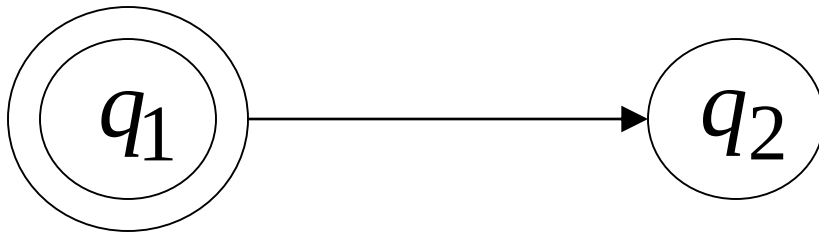
No possible transition

HALT!!!

Final States



Allowed



Not Allowed

- Final states have no outgoing transitions
- In a final state the machine halts

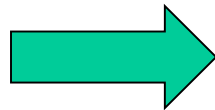
Acceptance

Accept Input



If machine halts
in a final state

Reject Input



If machine halts
in a non-final state

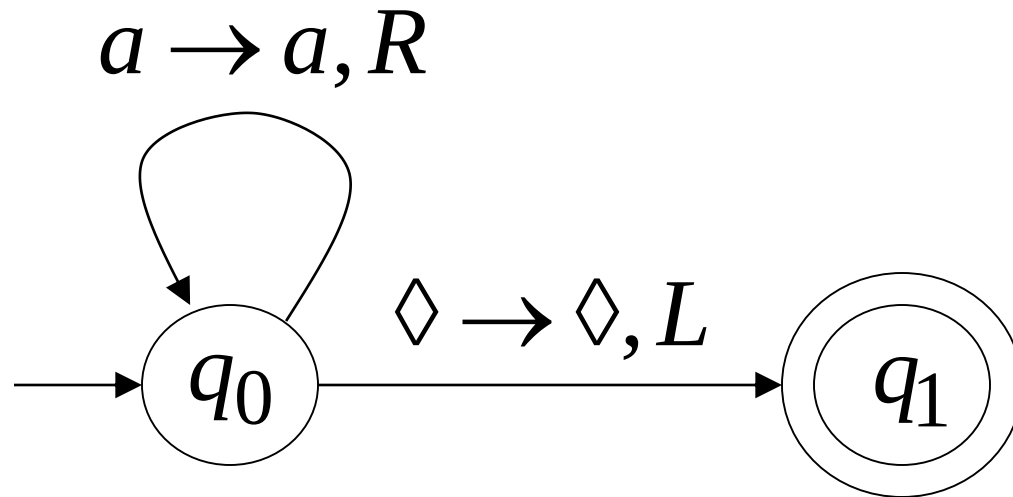
or

*[If machine enters
an infinite loop]*

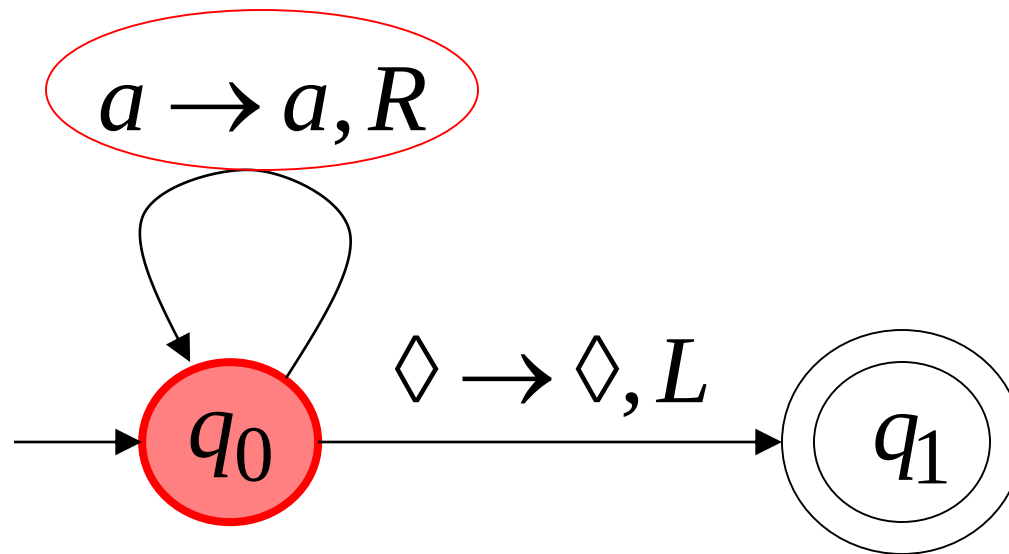
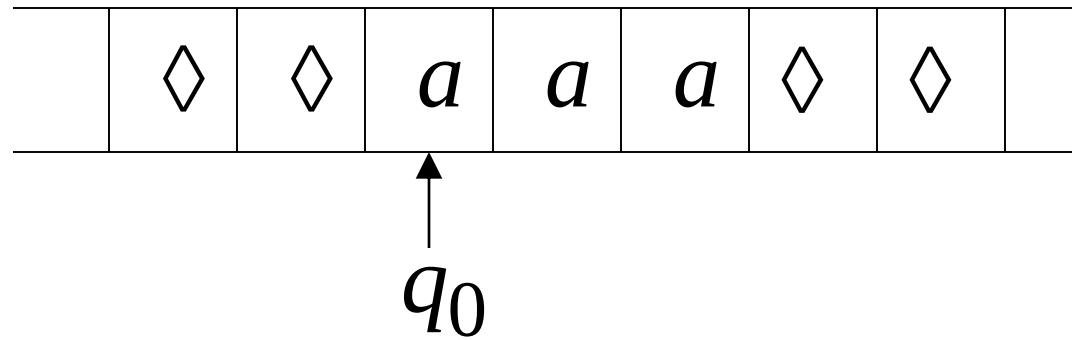
Turing Machine Example

A Turing machine that accepts the language:

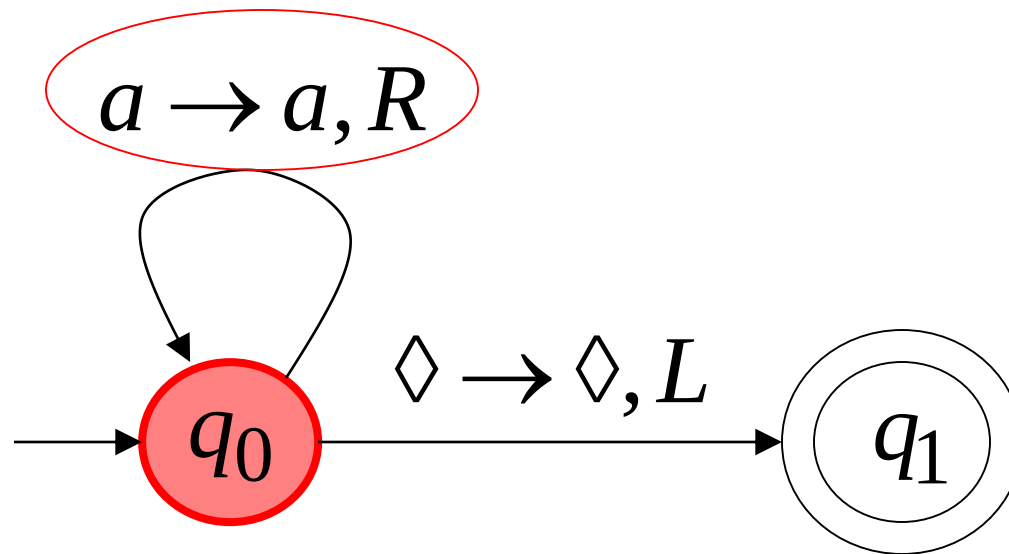
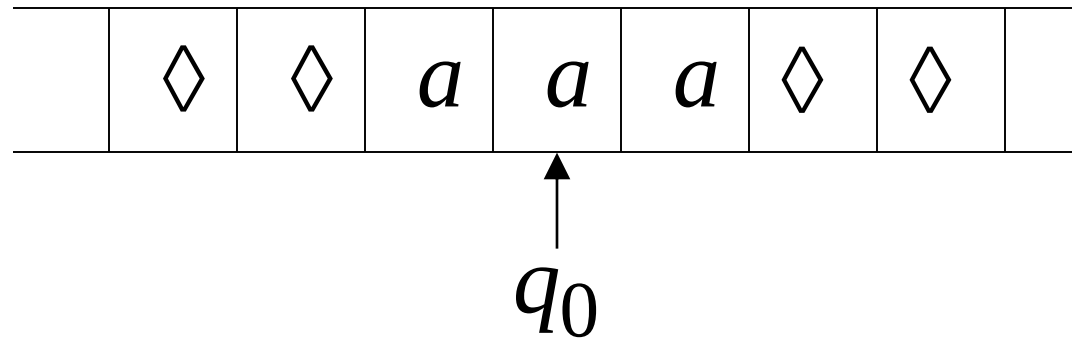
a^*



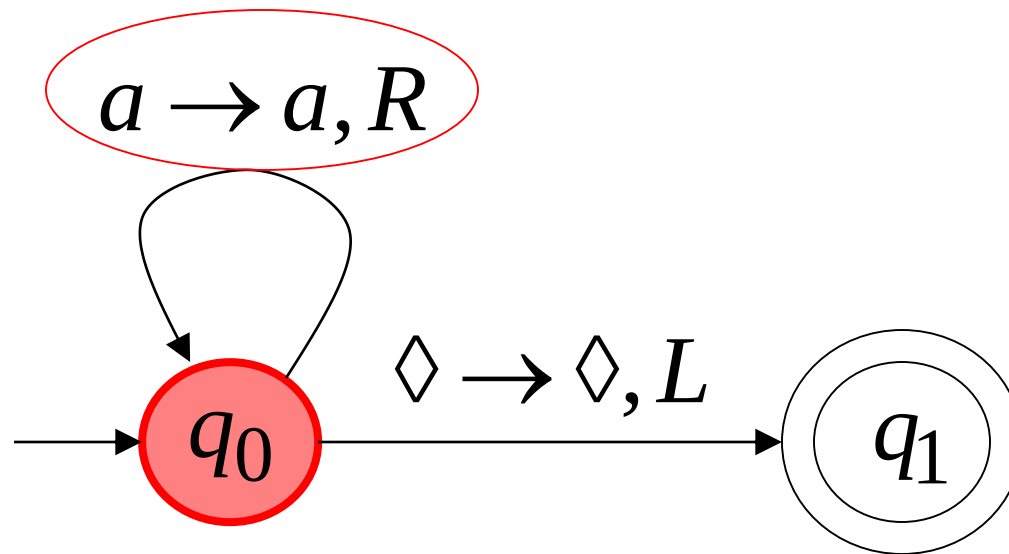
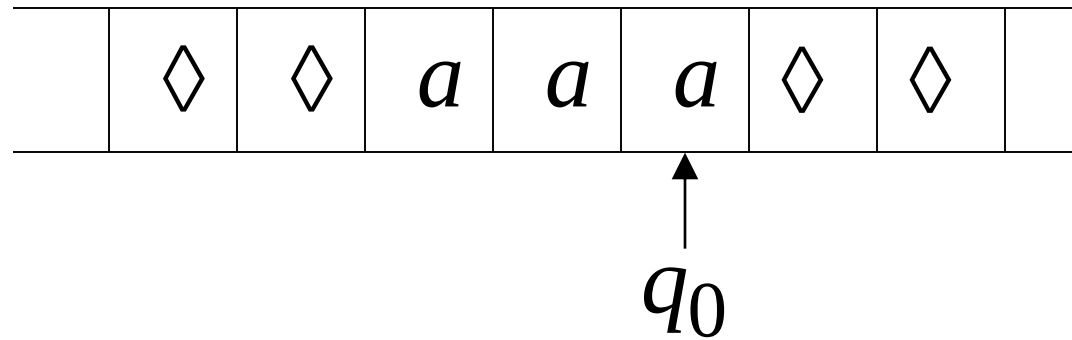
Time 0



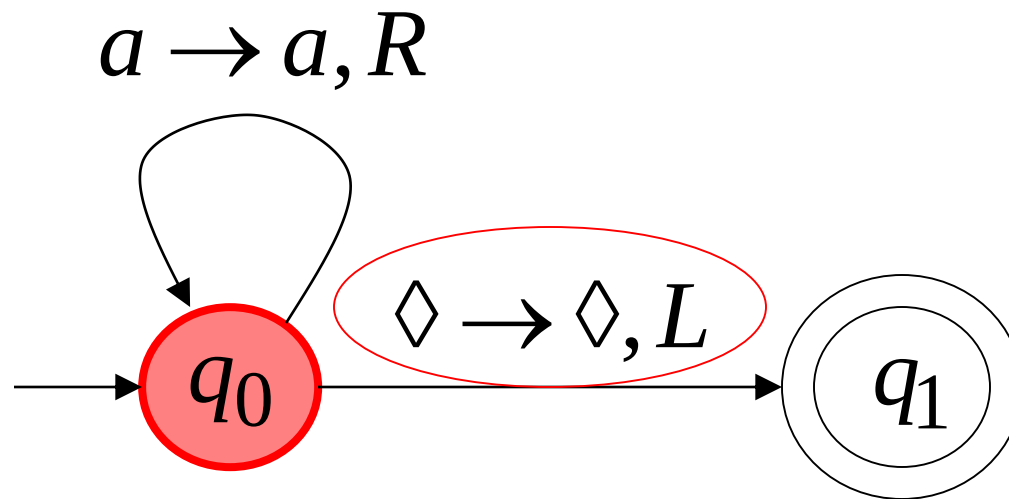
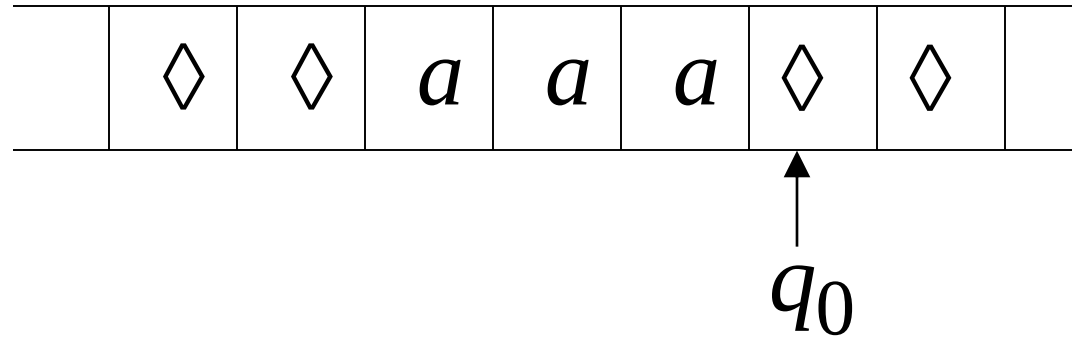
Time 1



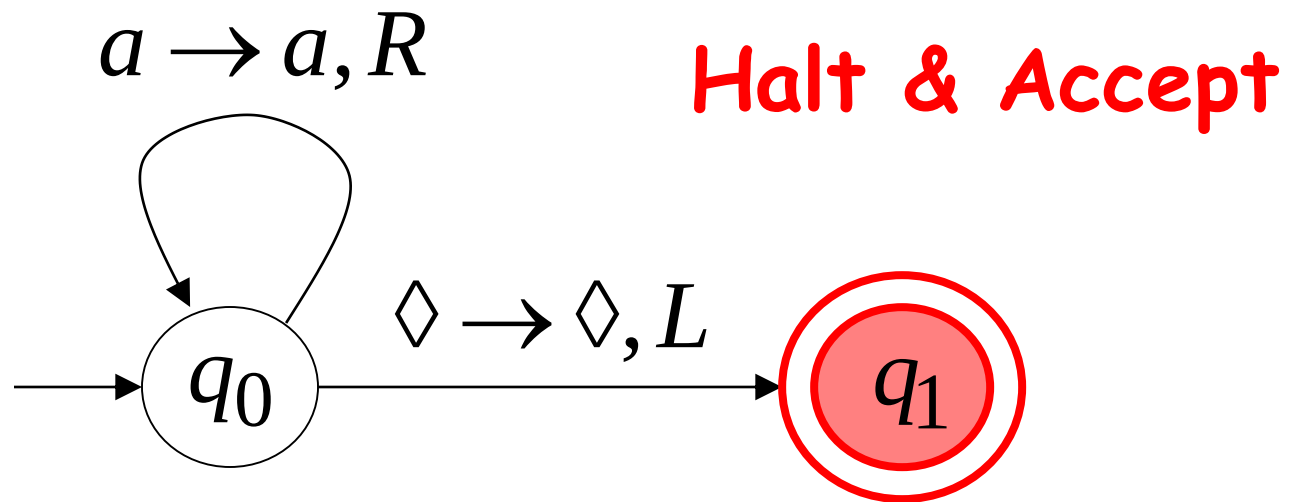
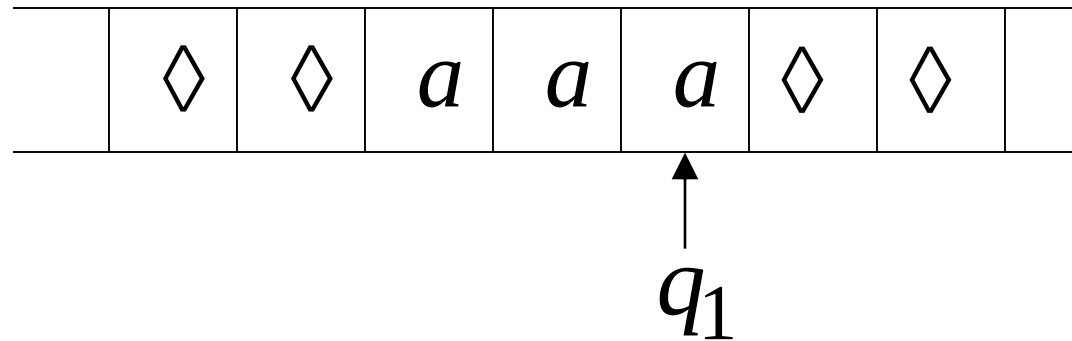
Time 2



Time 3

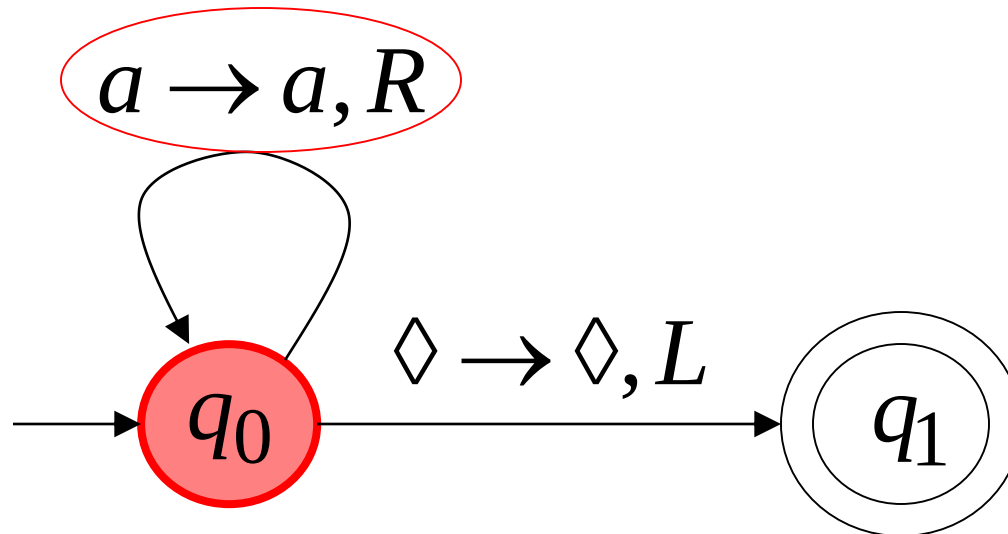
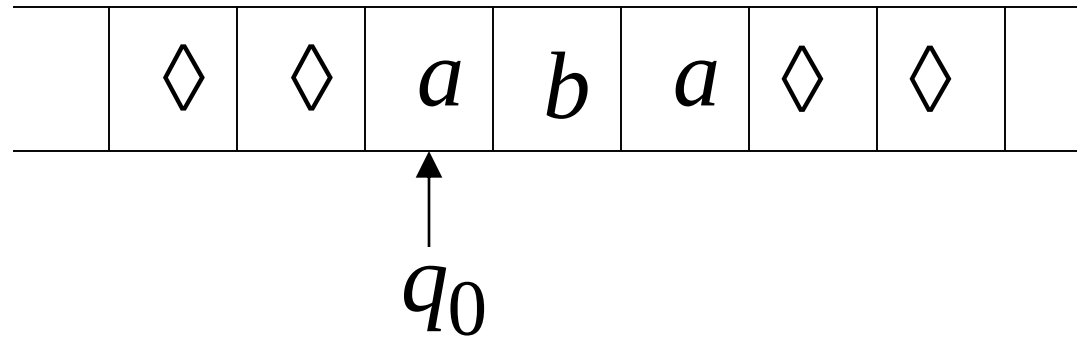


Time 4

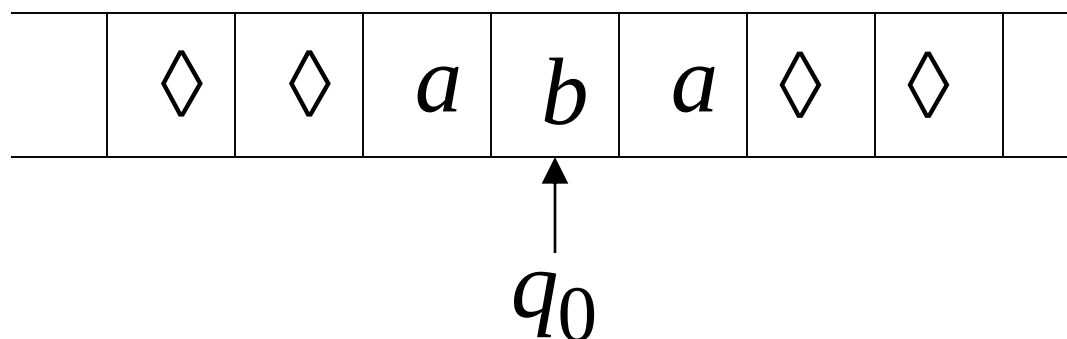


Rejection Example

Time 0

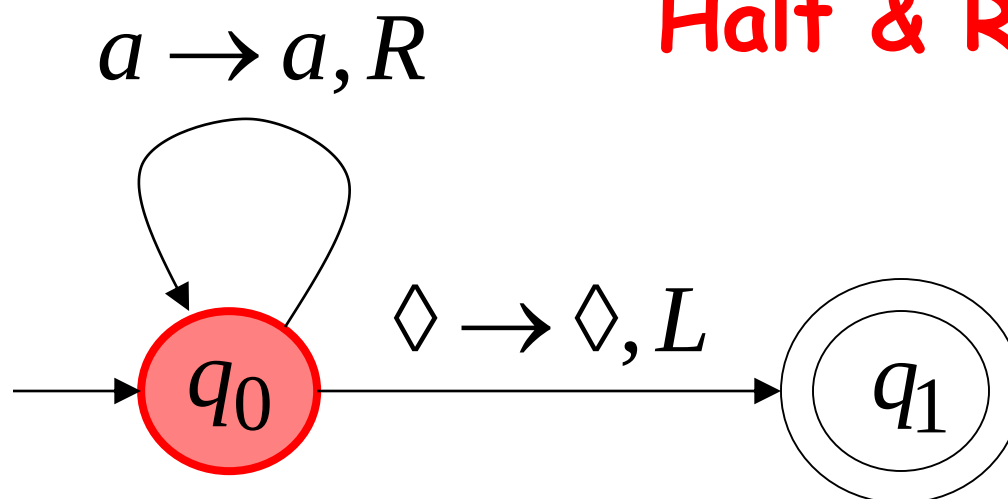


Time 1



No possible Transition

Halt & Reject



Draw TM that accept these languages.

1) $\{ \text{all strings in } \{a,b\}^* \text{ with prefix } ab \}$

2) $\{ \text{all binary strings containing substring } 001 \}$

3) $\{ \text{all binary strings without substring } 001 \}$

4) $\{a^n b : n \geq 0\}$

5) $\{awa : w \in \{a,b\}^*\}$

6) $\{ \}$

7) $\{a,b\}^*$

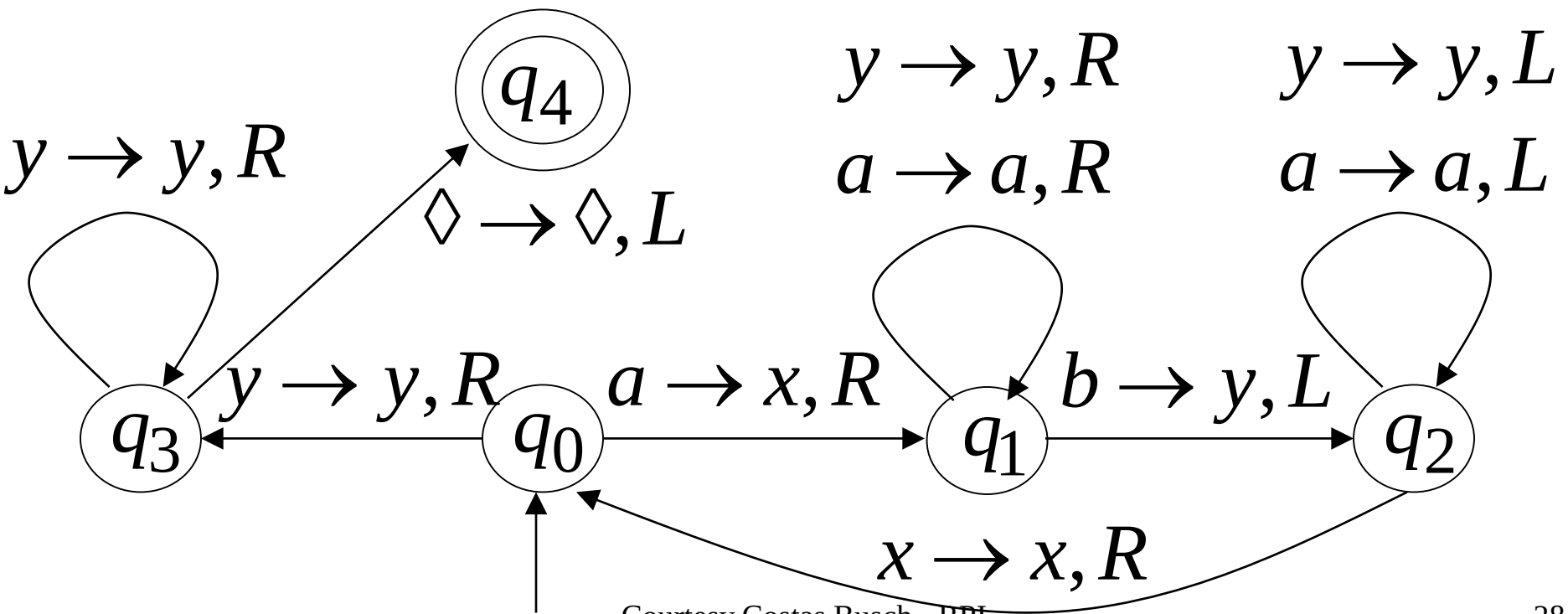
8) $\{\lambda\}$

9) $\{x : x \in \{a\}^* \text{ and } |x| \text{ is even}\}$

--DO--

Another Turing Machine Example

Turing machine for the language $\{a^n b^n\}$



Basic Idea:

Match **a**'s with **b**'s:

Repeat:

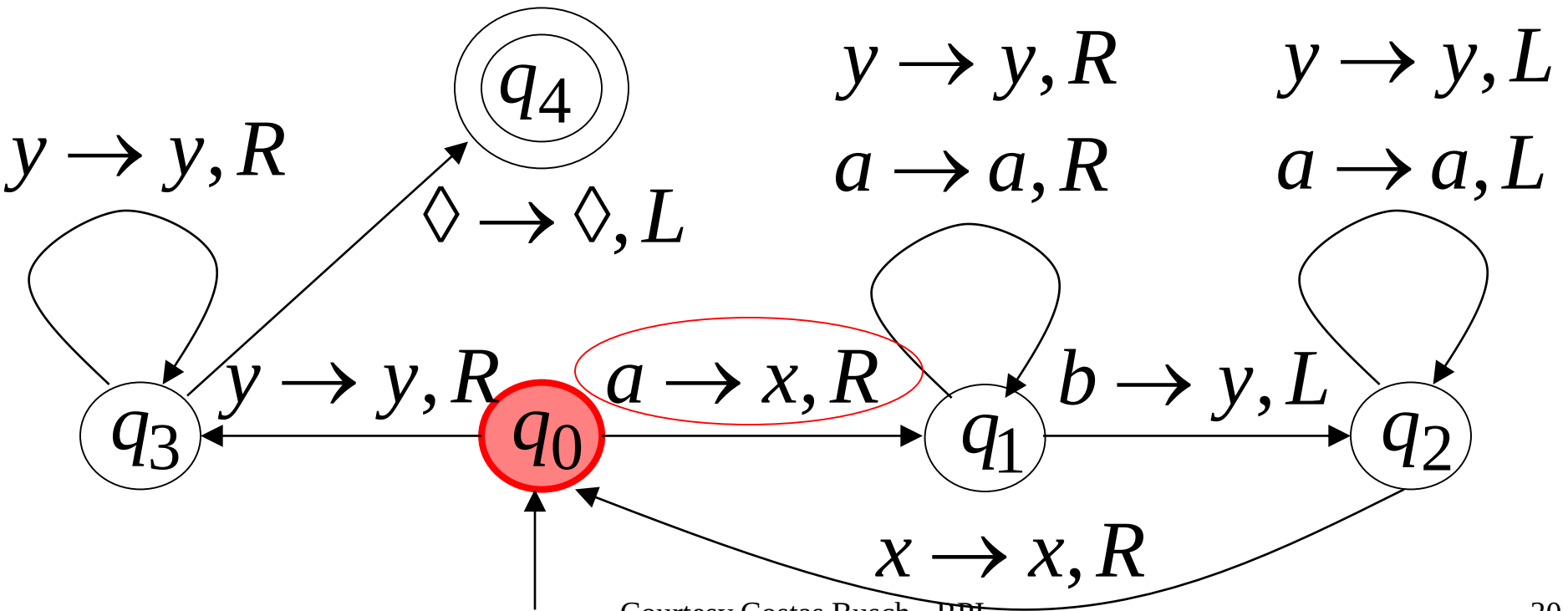
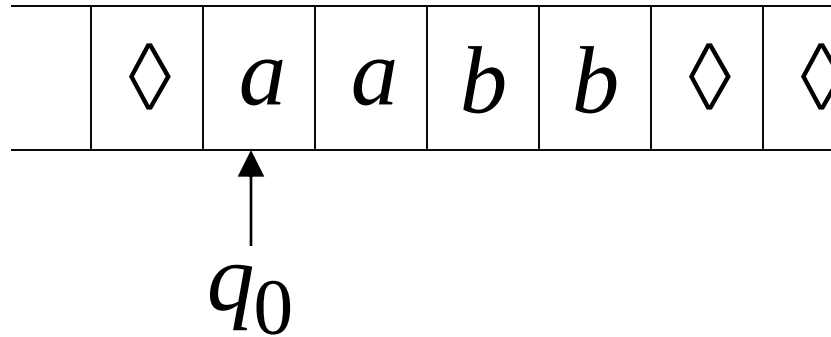
replace leftmost **a** with **x**

find leftmost **b** and replace it with **y**

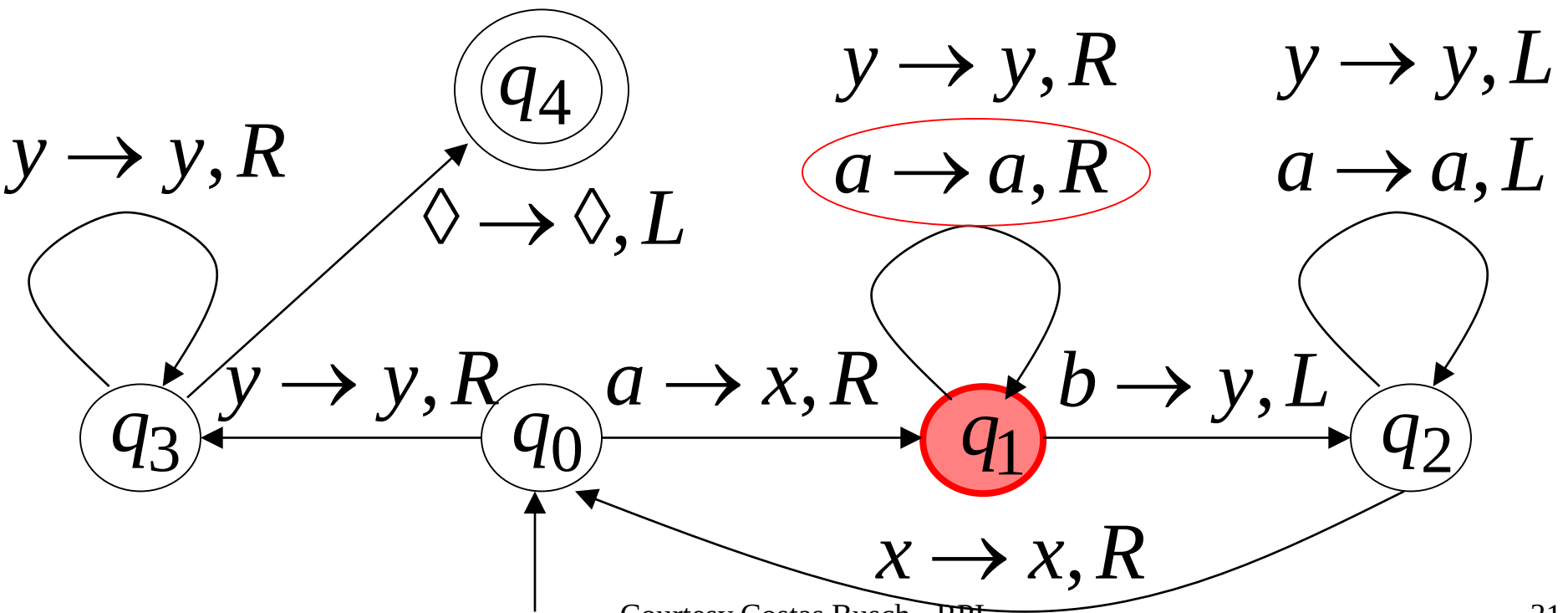
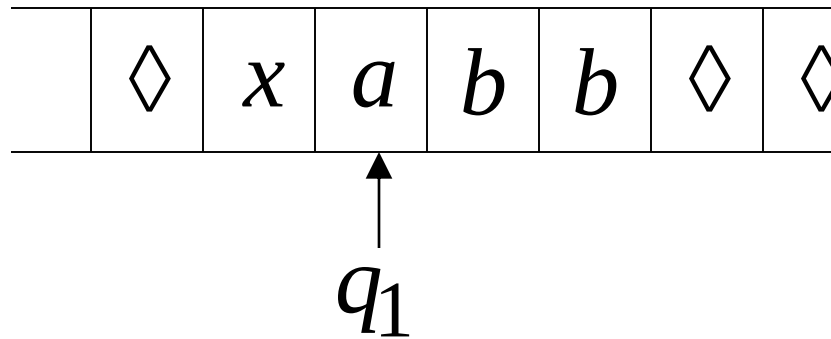
Until there are no more **a**'s or **b**'s

If there is a remaining **a** or **b** reject

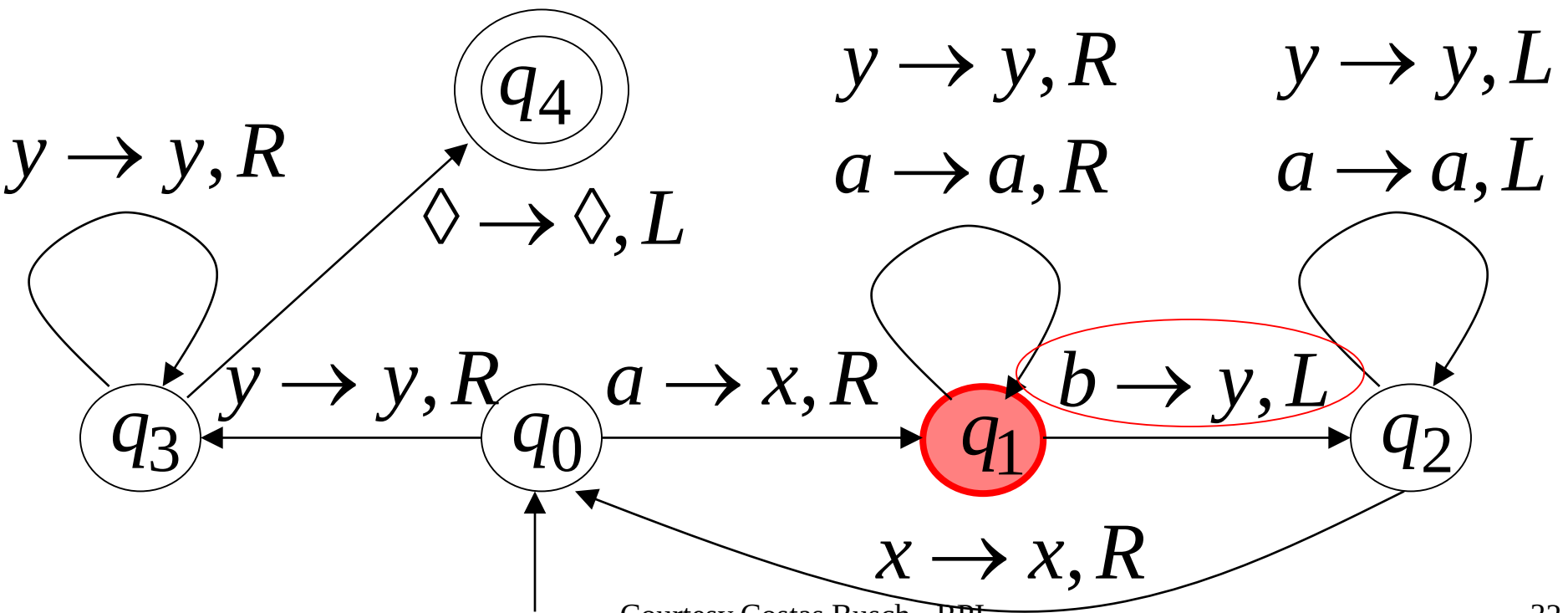
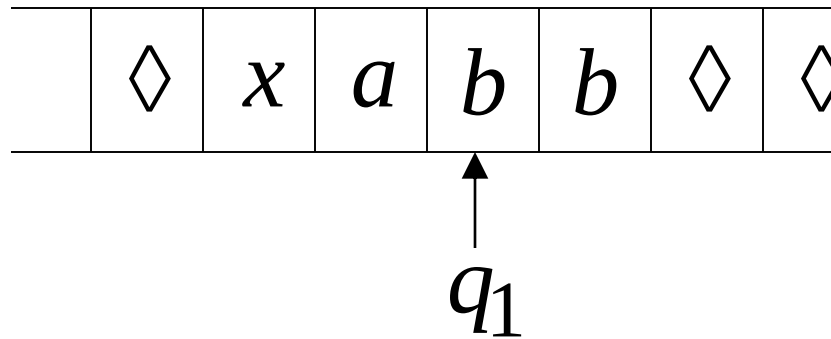
Time 0



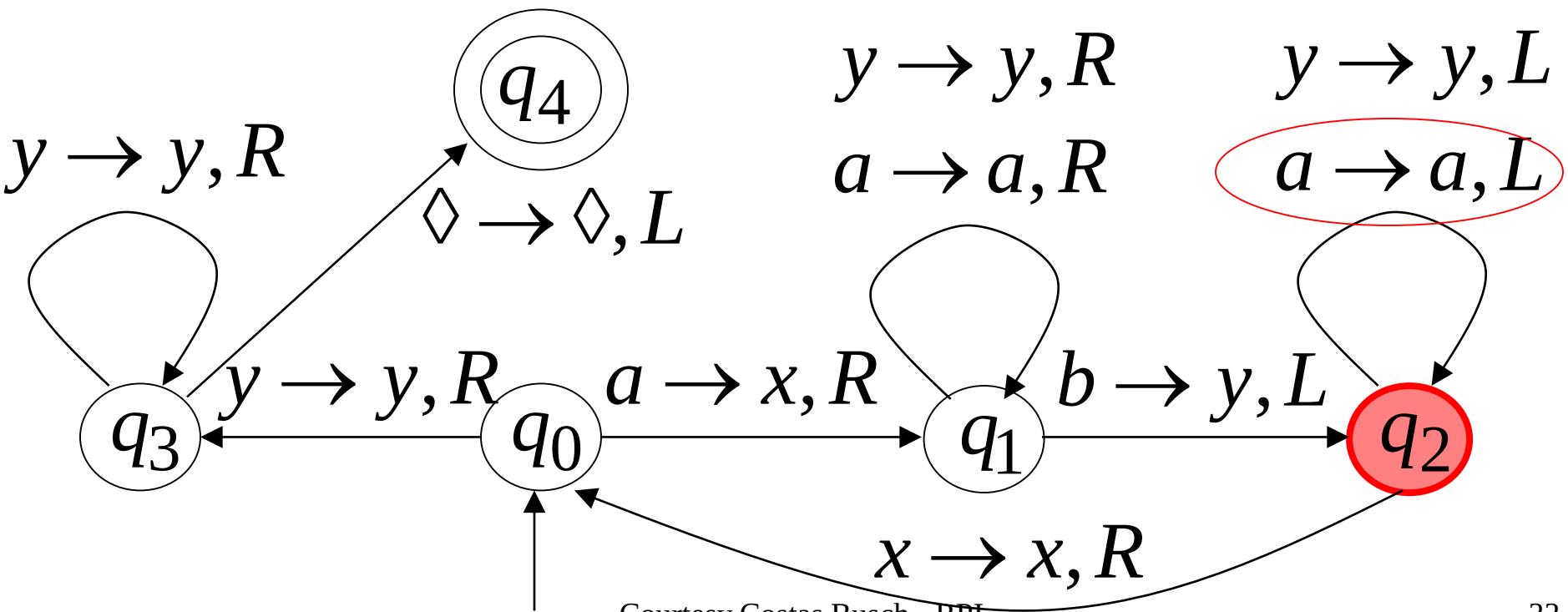
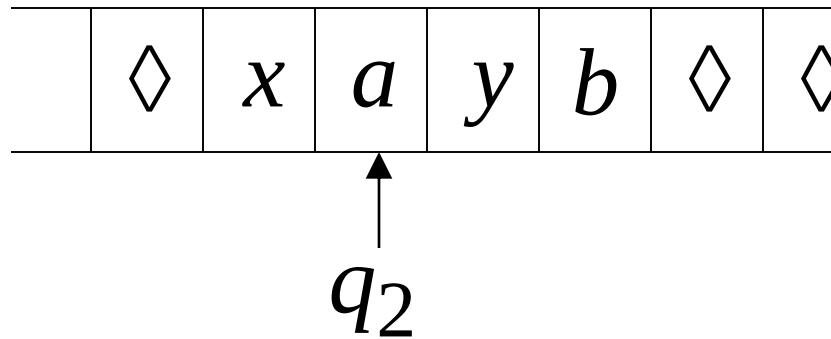
Time 1



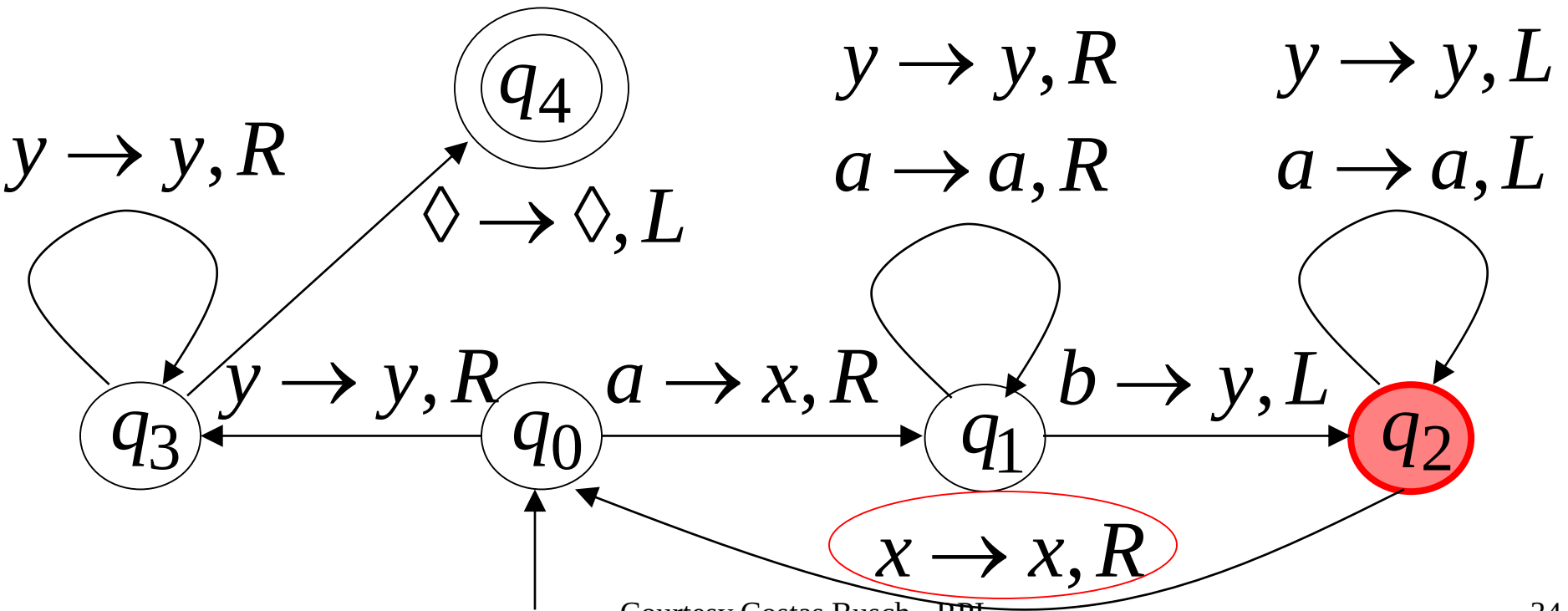
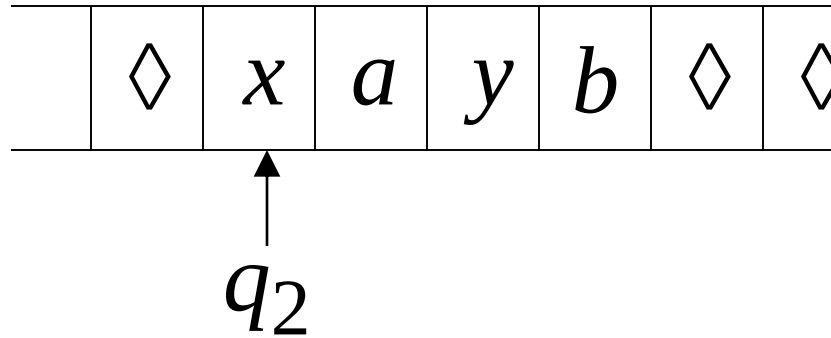
Time 2



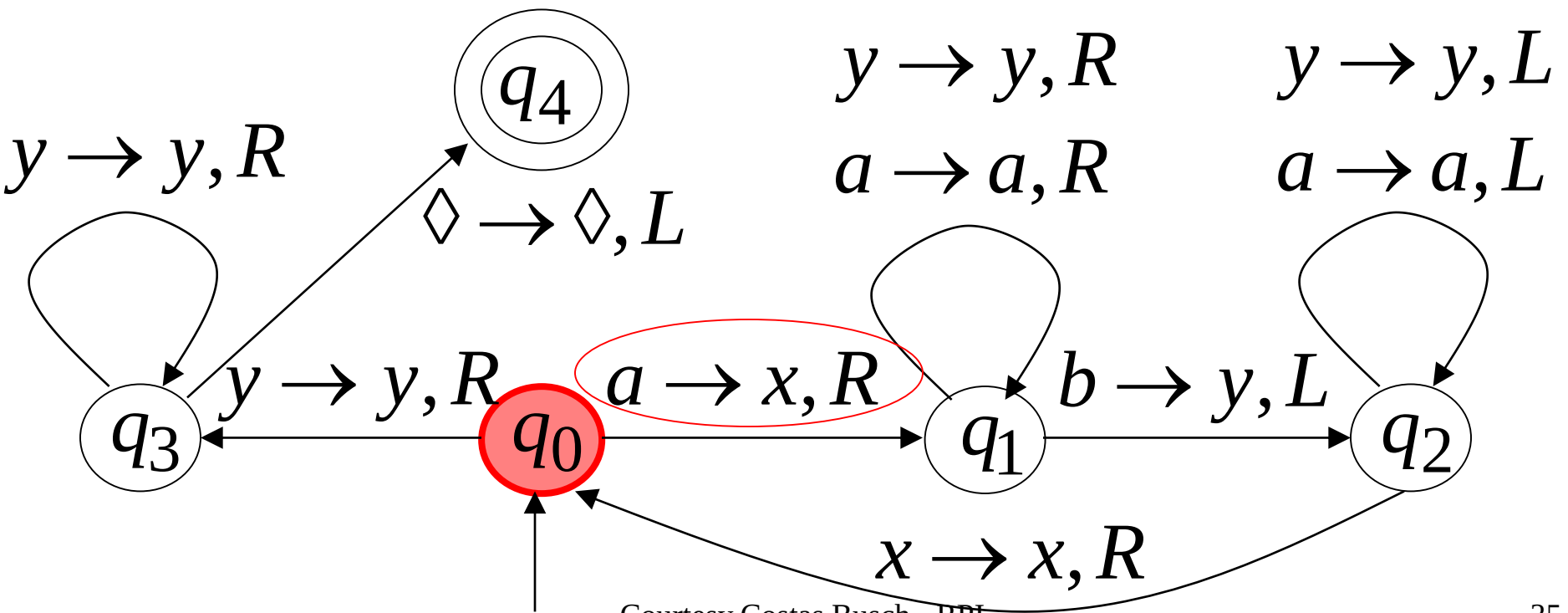
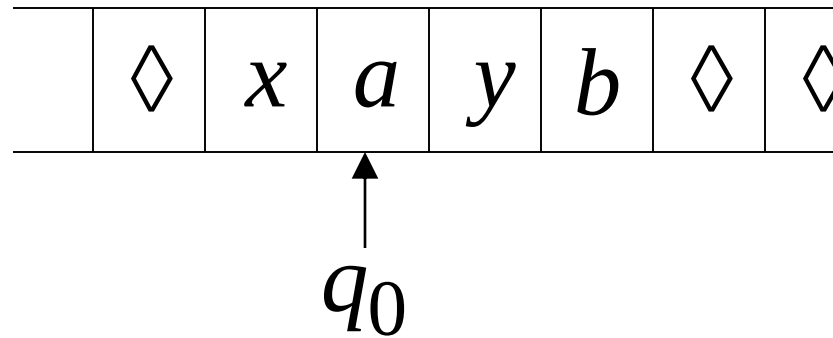
Time 3



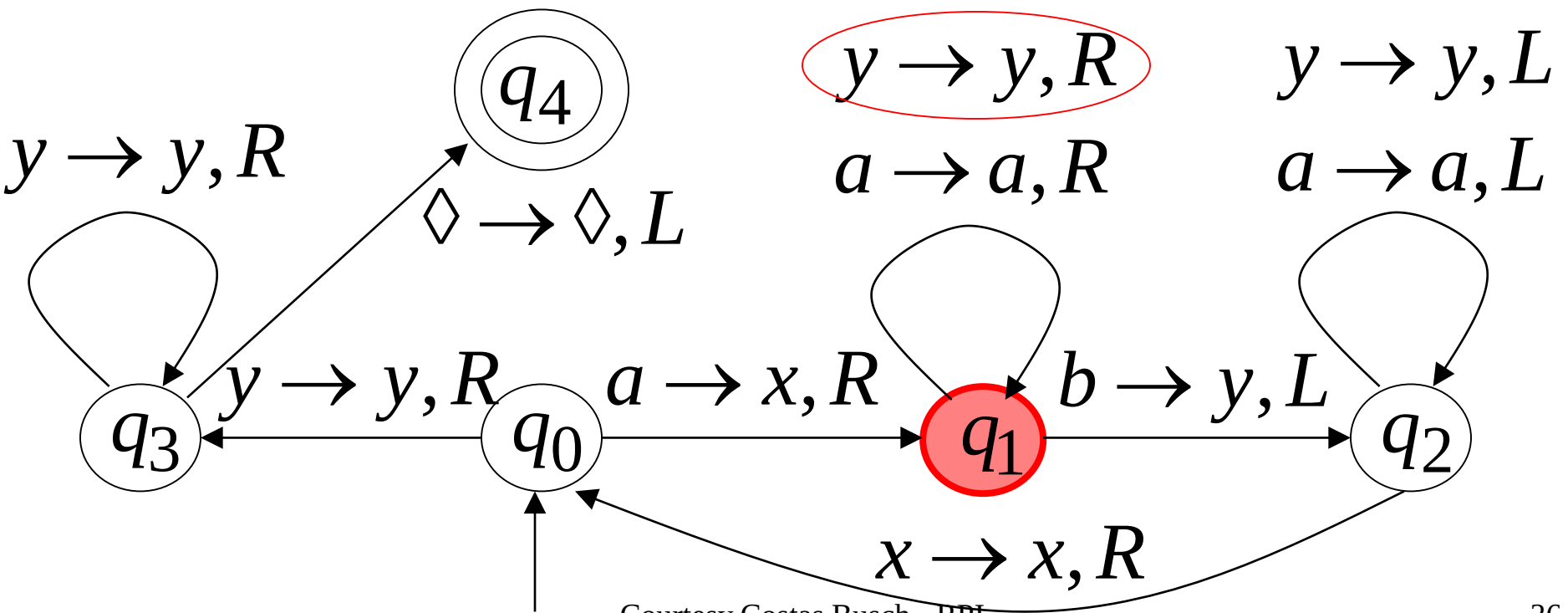
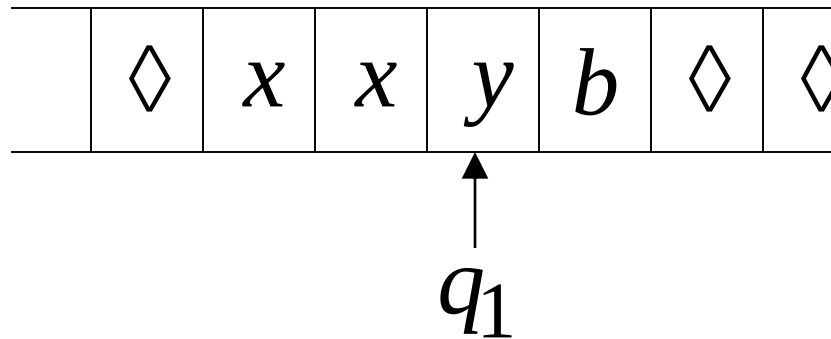
Time 4



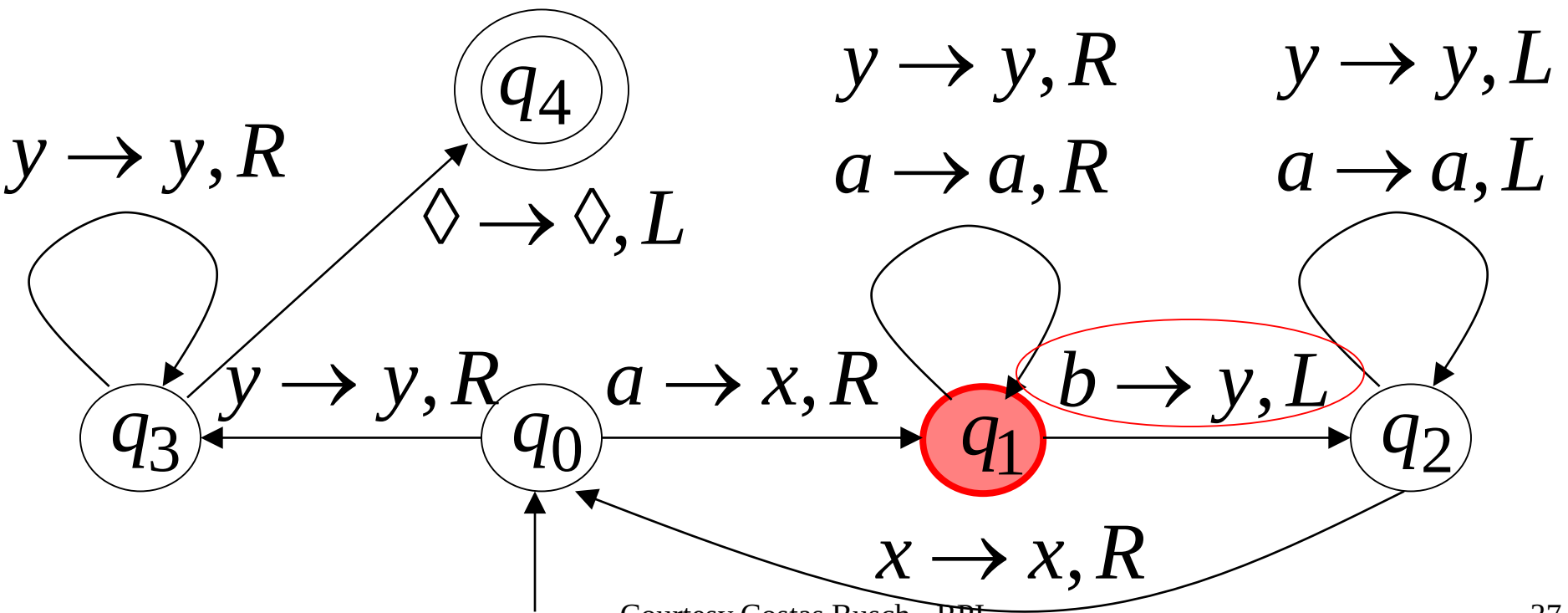
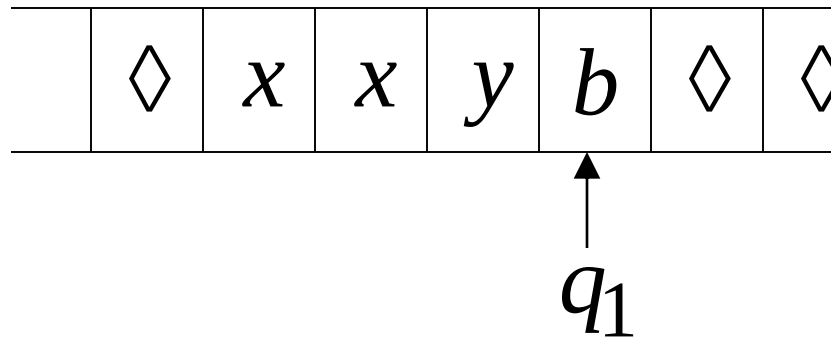
Time 5



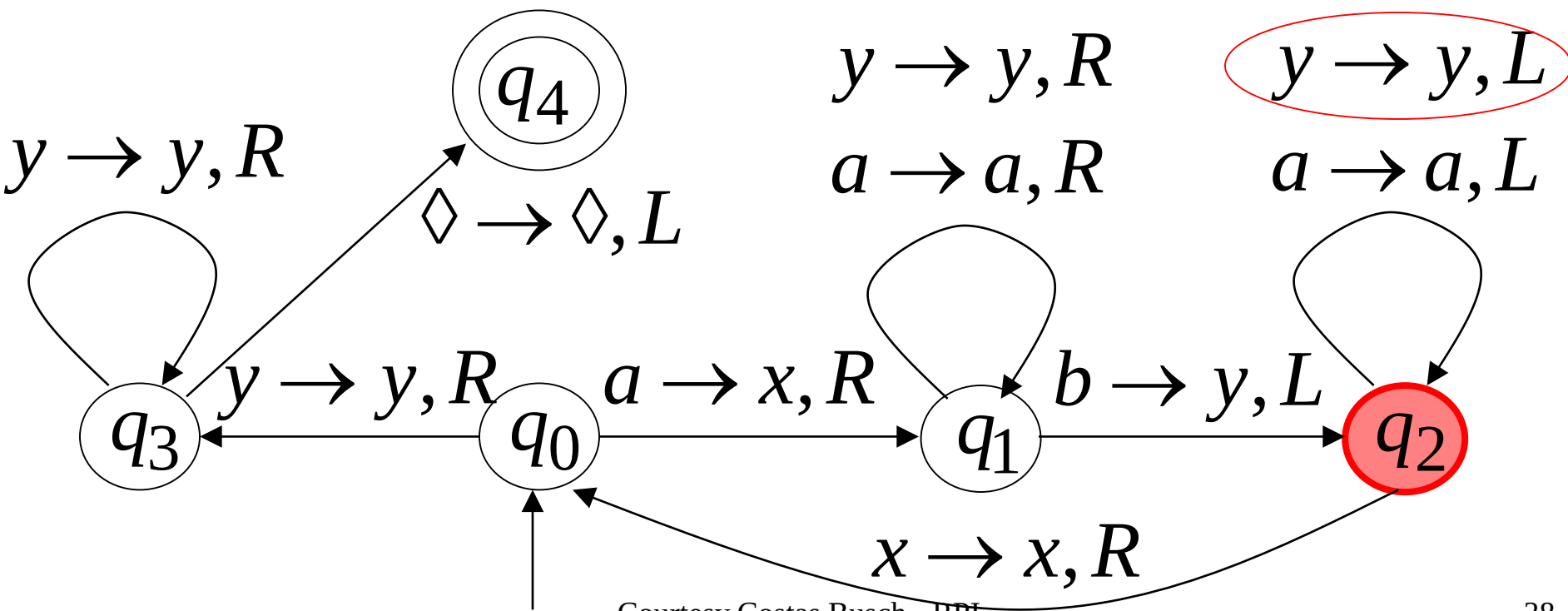
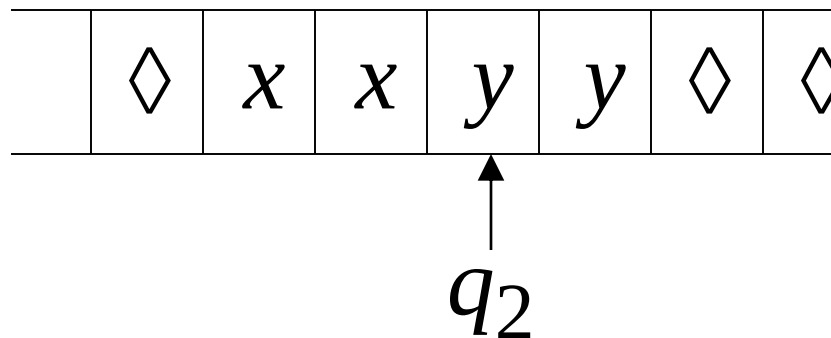
Time 6



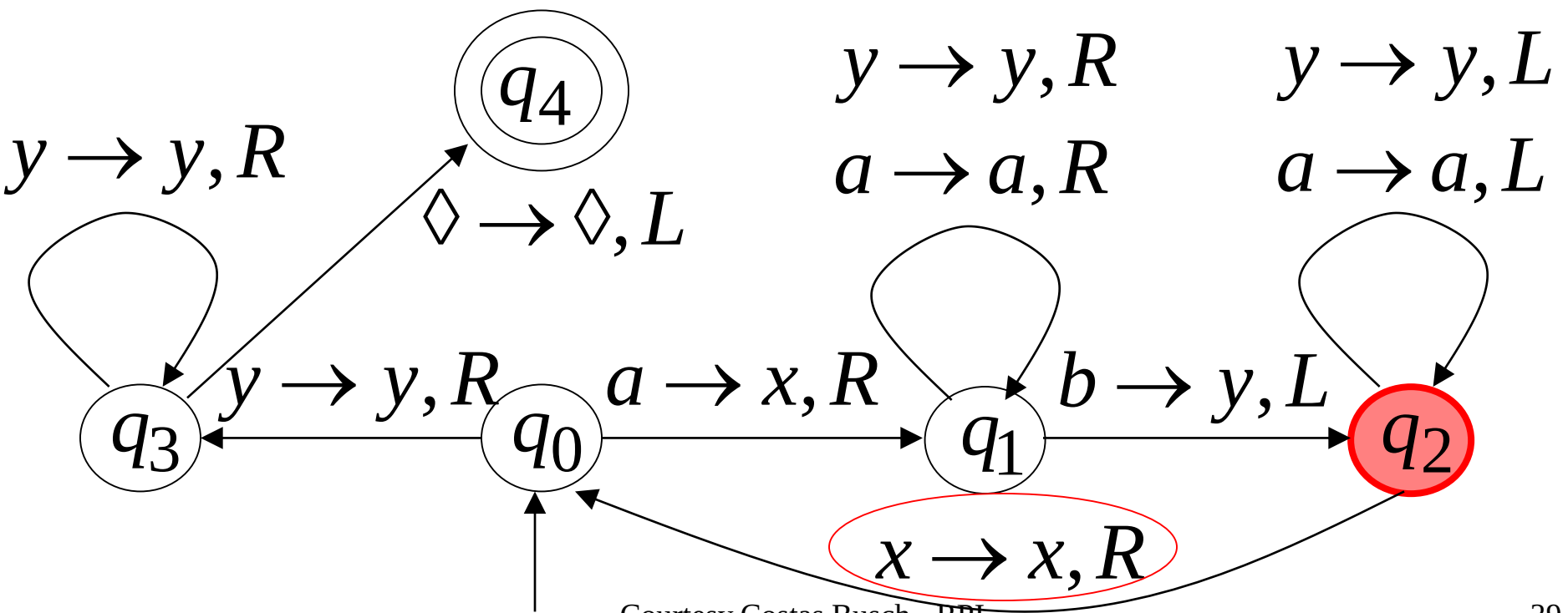
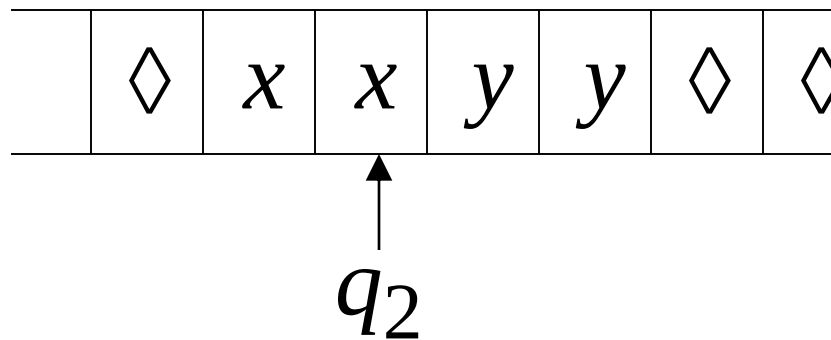
Time 7



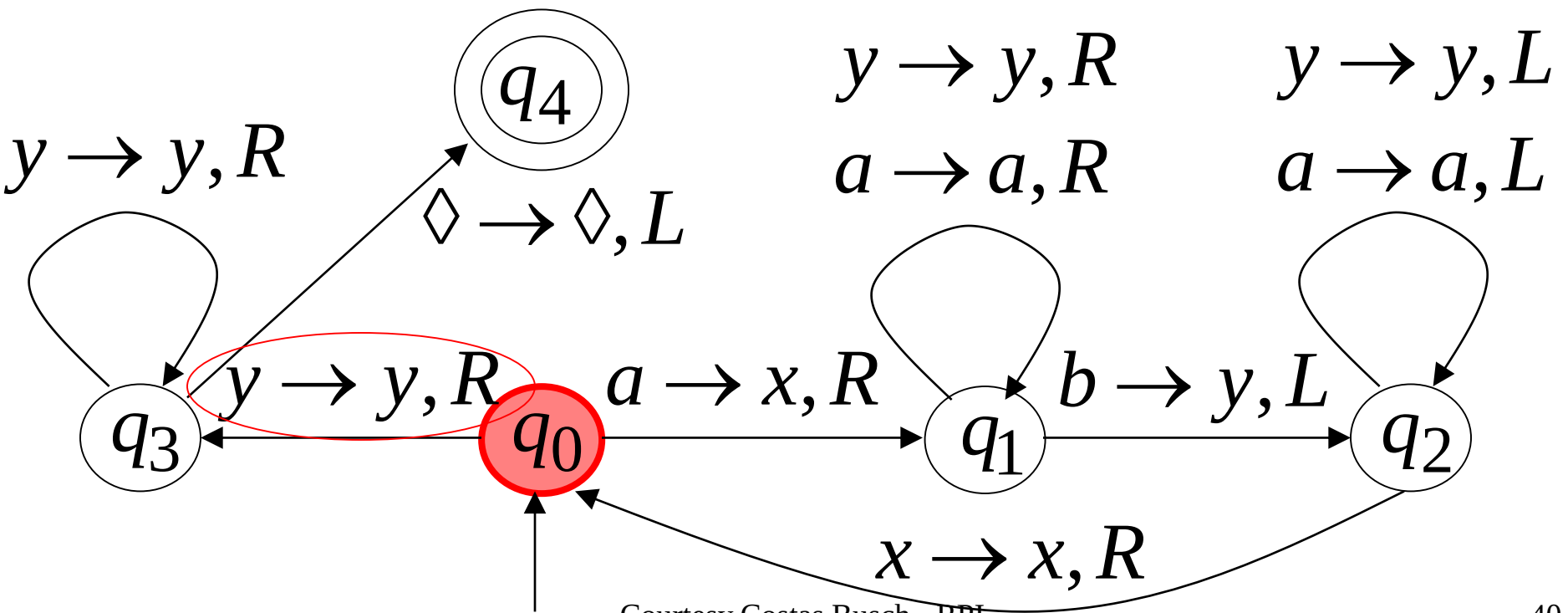
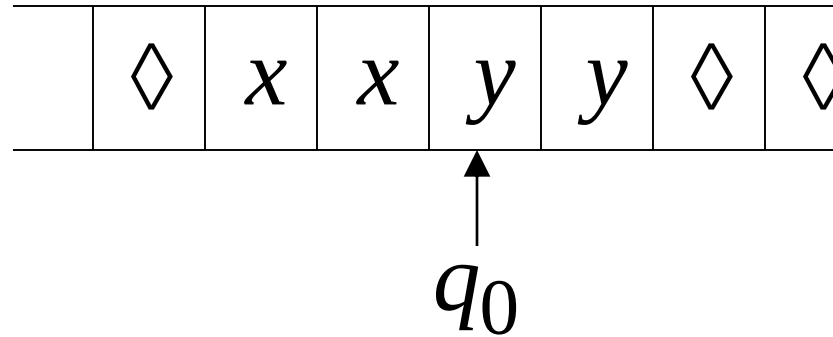
Time 8



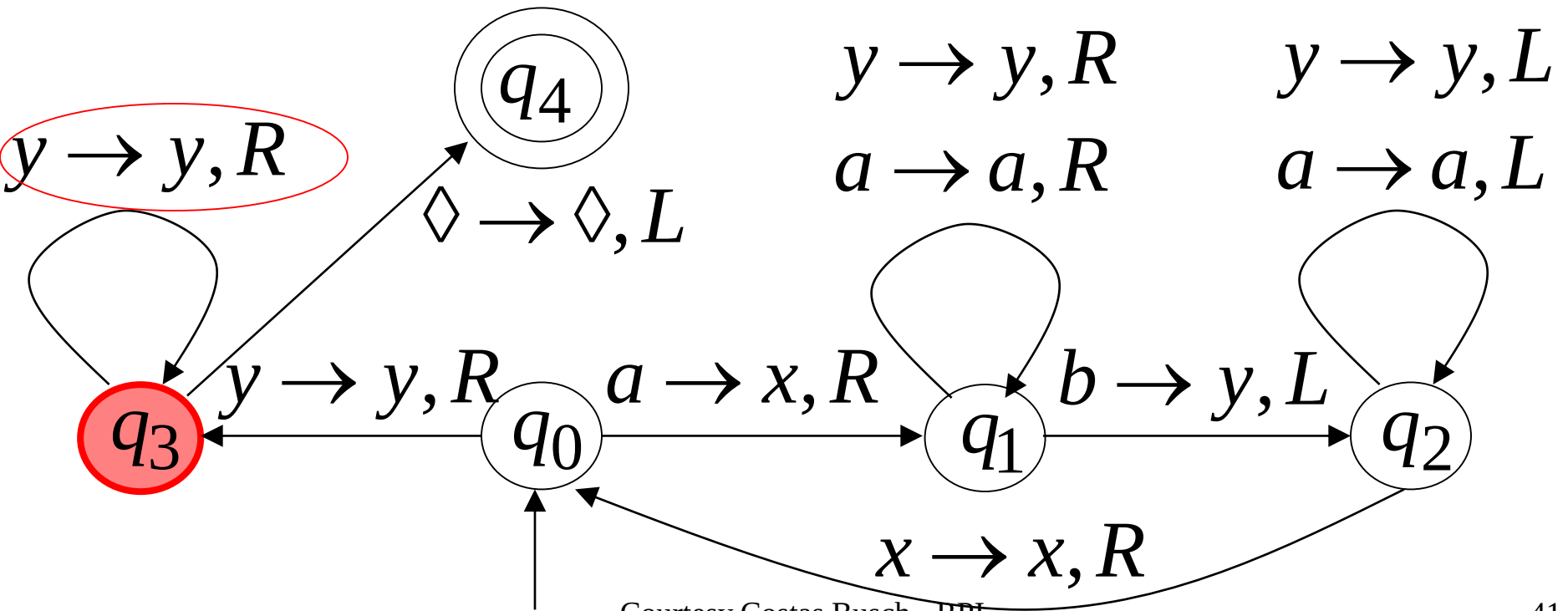
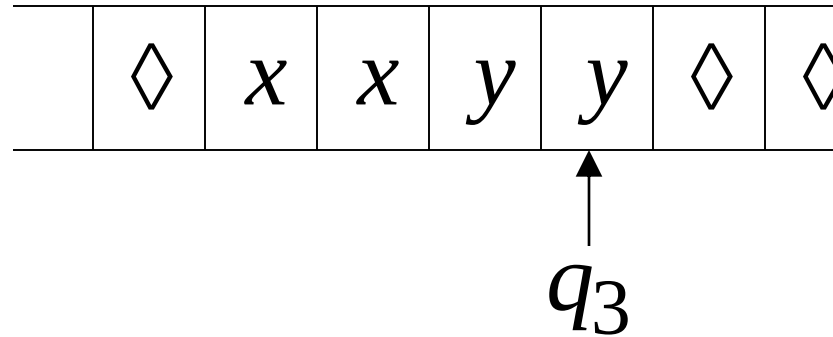
Time 9



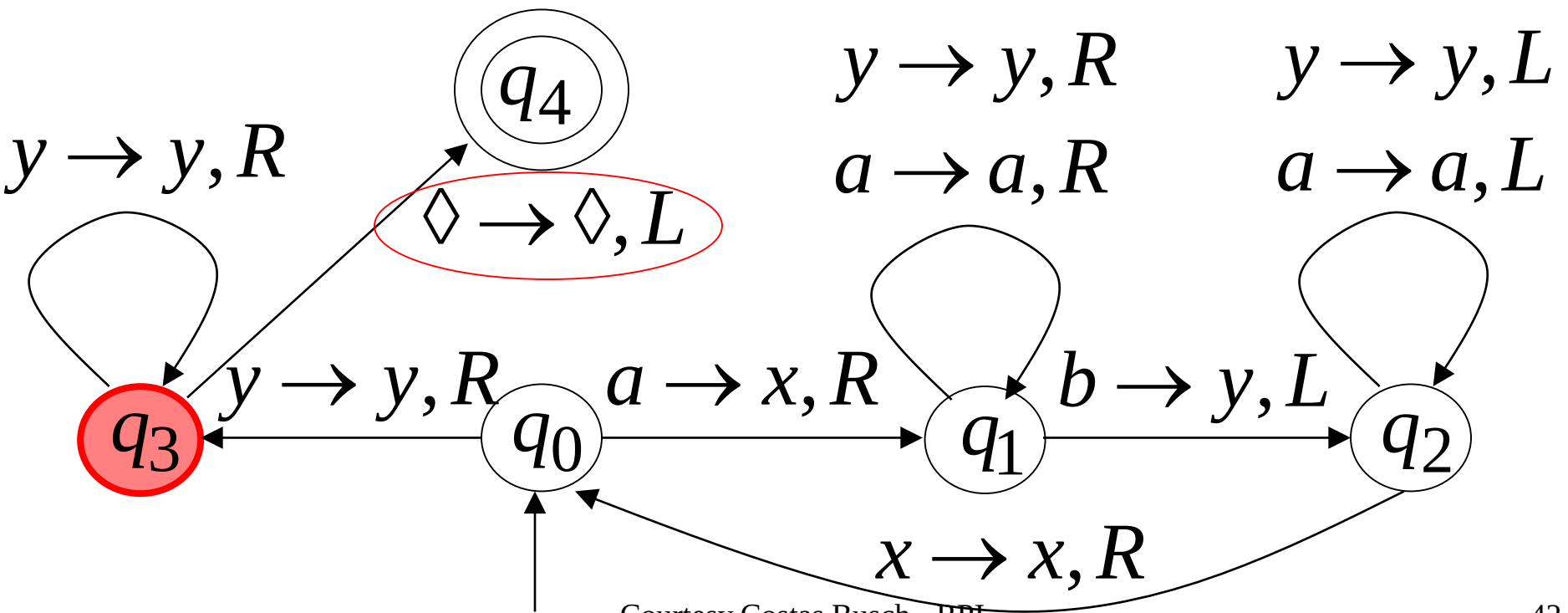
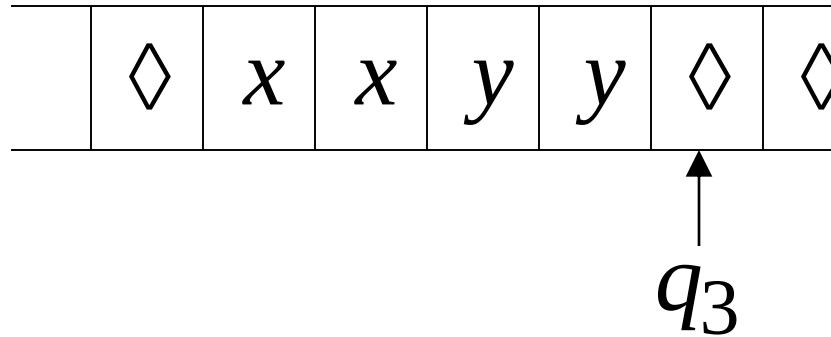
Time 10



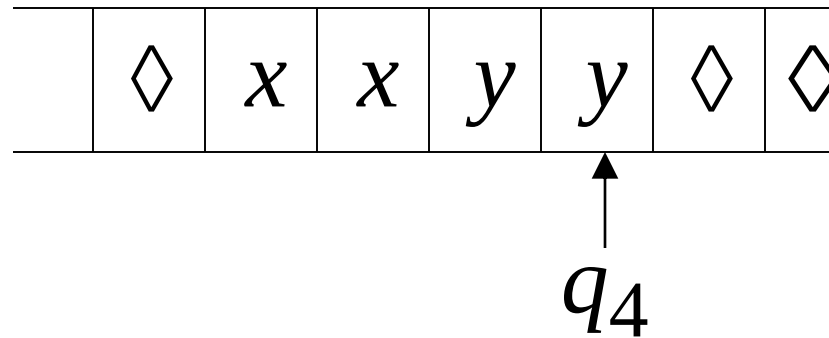
Time 11



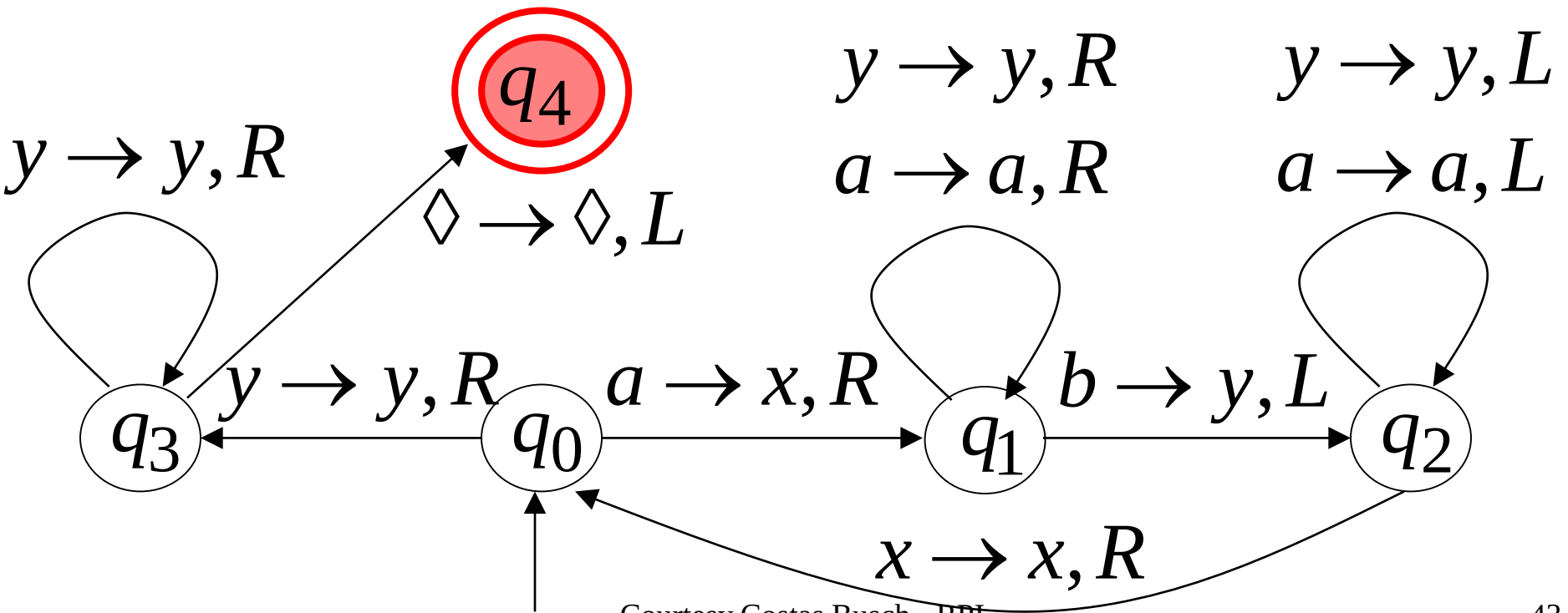
Time 12



Time 13



Halt & Accept

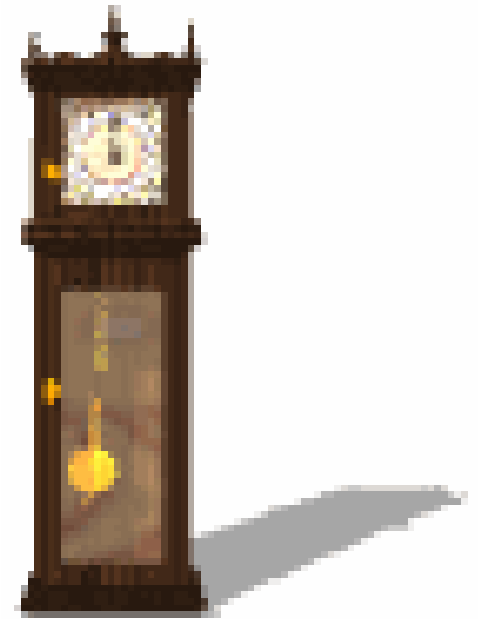


Observation:

If we modify the
machine for the language $\{a^n b^n\}$

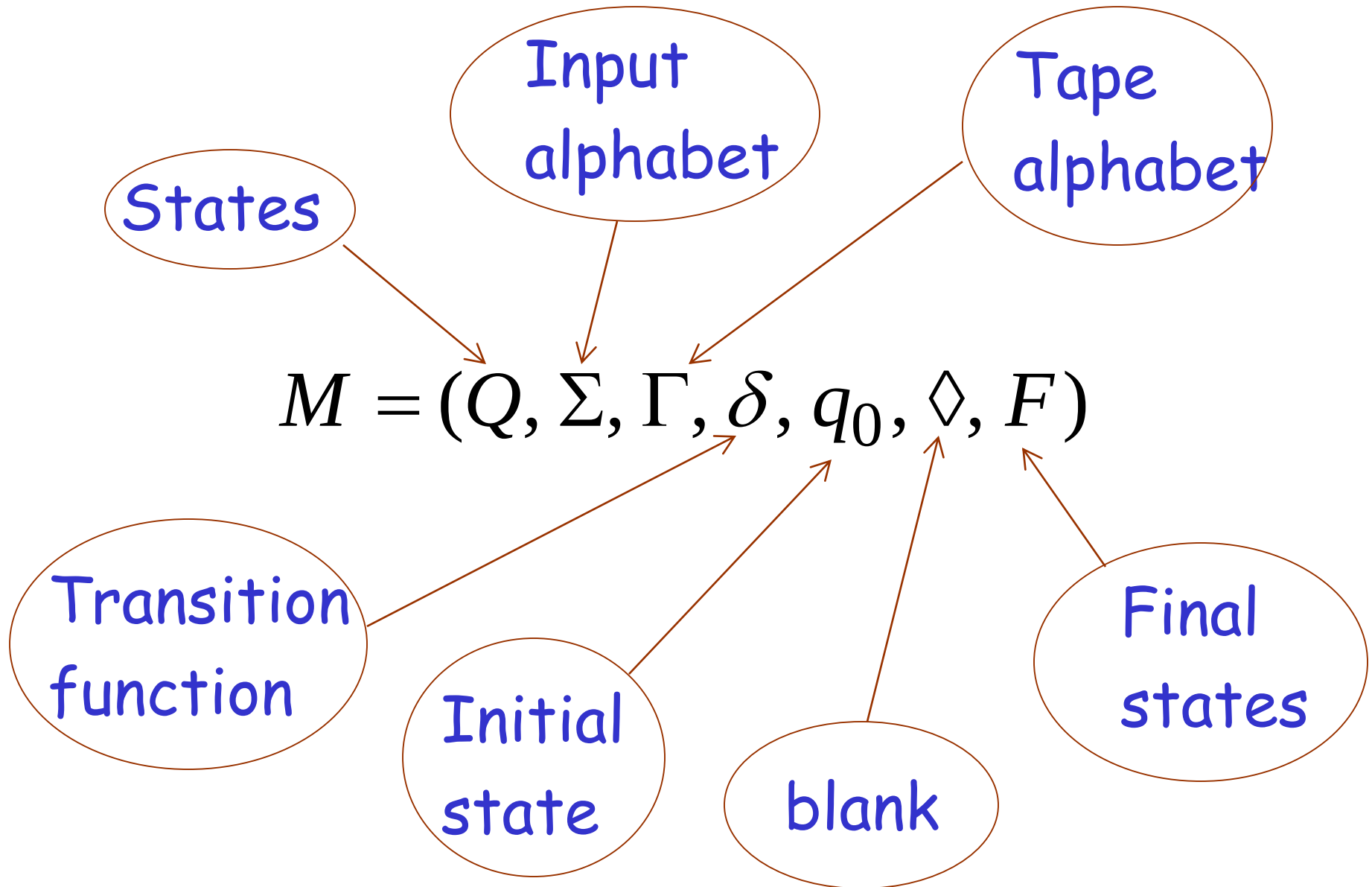
we can easily construct
a machine for the language $\{a^n b^n c^n\}$

Activity Time



Write basic idea of TM for
 $\{v.v: v \in \{a,b\}^*\}$

Turing Machine:



Turing Machine:

$$M = (Q, \Sigma, \Gamma, \delta, q_0, \diamond, F)$$

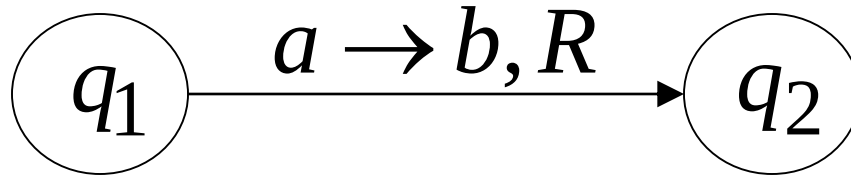
$$\Sigma \subseteq \Gamma$$

$$\diamond \in \Gamma, \diamond \notin \Sigma$$

$$q_0 \in Q$$

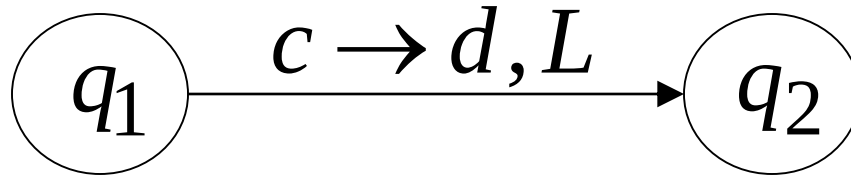
$$F \subset Q$$

Transition Function



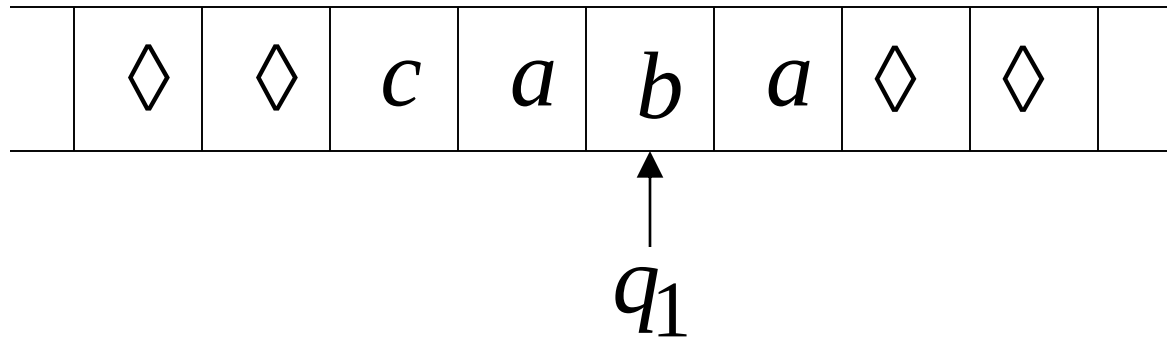
$$\delta(q_1, a) = (q_2, b, R)$$

Transition Function



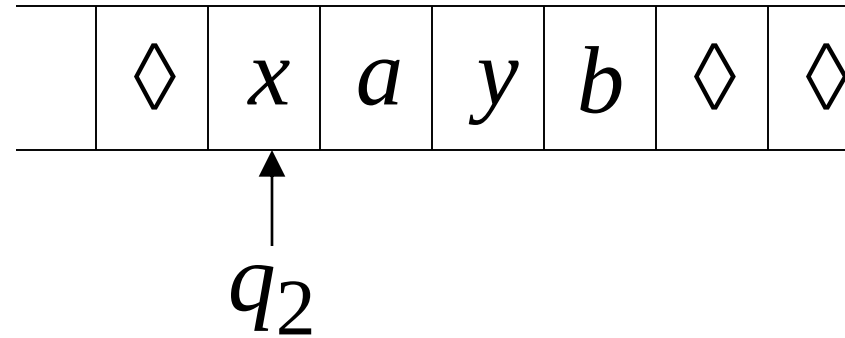
$$\delta(q_1, c) = (q_2, d, L)$$

Configuration

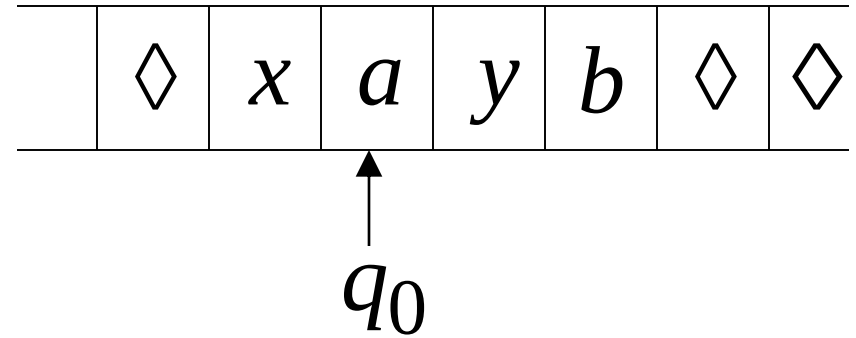


Instantaneous description: $ca\ q_1\ ba$

Time 4

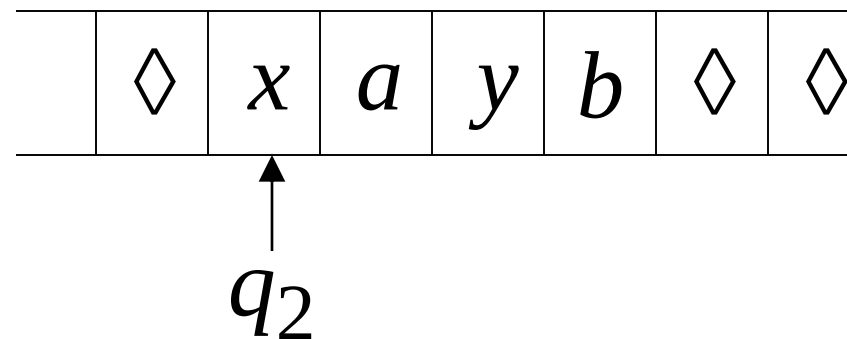


Time 5

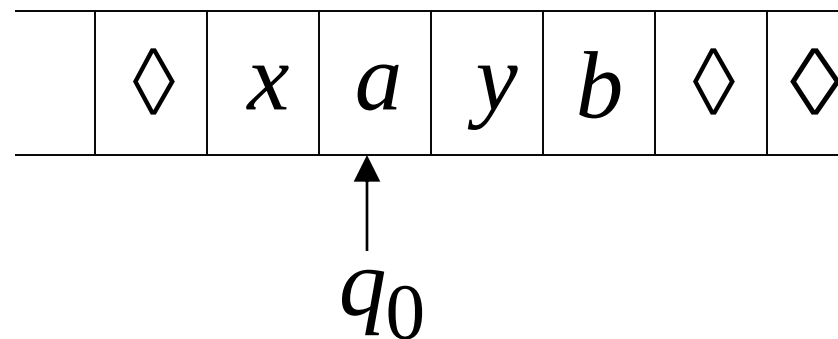


A Move: $q_2 \ x a y b \succ x \ q_0 \ a y b$

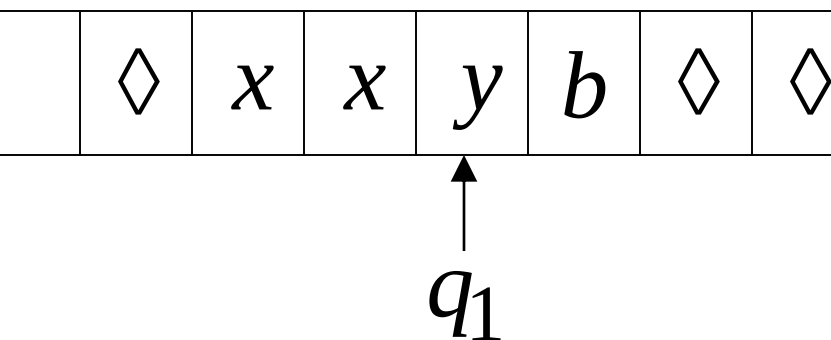
Time 4



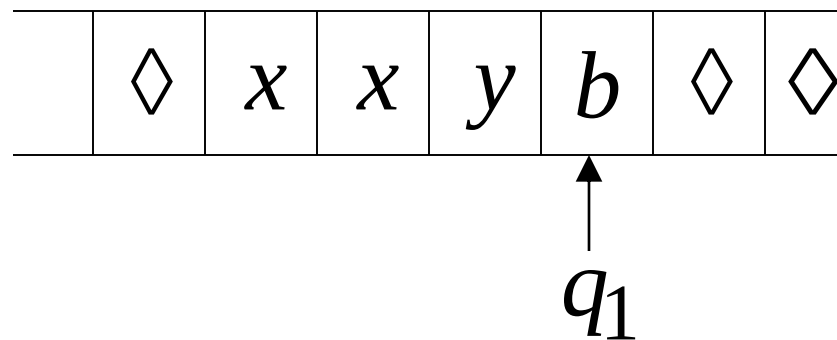
Time 5



Time 6



Time 7



A computation

$q_2 \ x a y b \succ x \ q_0 \ a y b \succ x x \ q_1 \ y b \succ x x y \ q_1 \ b$

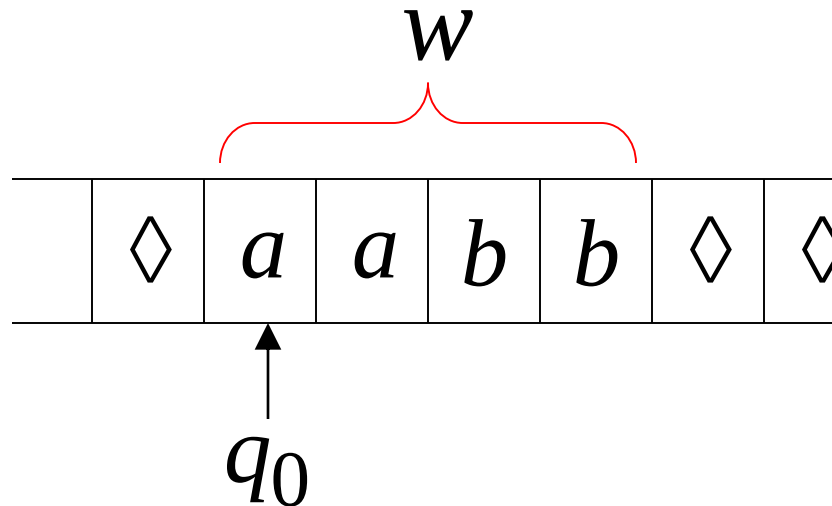
$$q_2 \ x a y b \succ x \ q_0 \ a y b \succ x x \ q_1 \ y b \succ x x y \ q_1 \ b$$

Equivalent notation:

$$q_2 \ x a y b \overset{*}{\succ} x x y \ q_1 \ b$$

Initial configuration: $q_0 w$

Input string

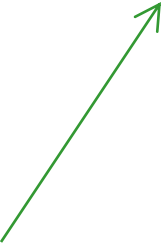


The Accepted Language

For any Turing Machine M

$$L(M) = \{w : q_0 w \xrightarrow{*} x_1 q_f x_2\}$$

Initial state



Final state



If a language L is accepted
by a Turing machine M
then we say that L is:

- Turing Recognizable

Other names used:

- Turing Acceptable
- Recursively Enumerable

Standard Turing Machine

The machine we described is the standard:

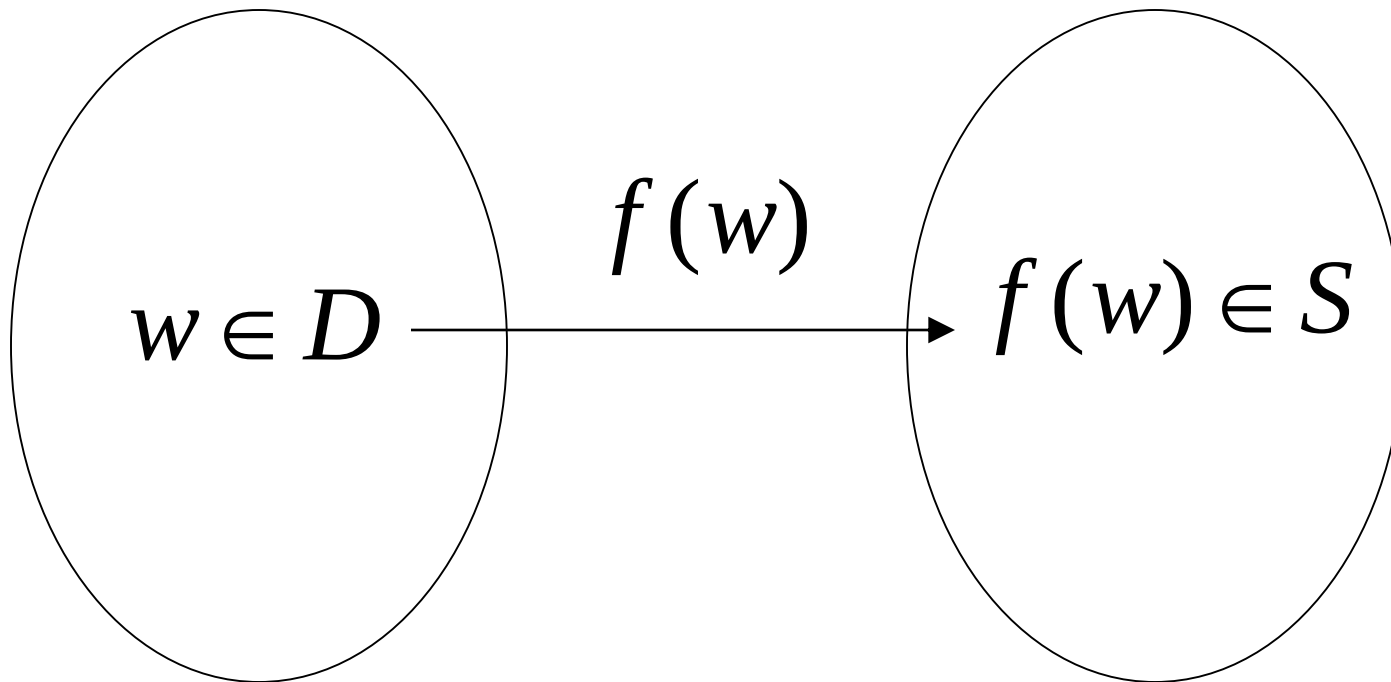
- Deterministic
- Infinite tape in both directions
- Tape is the input/output file

Computing Functions with Turing Machines

A function $f(w)$ has:

Domain: D

Result Region: S



A function may have many parameters:

Example: Addition function

$$f(x, y) = x + y$$

Integer Domain

Decimal: 5

Binary: 101

Unary: 1111

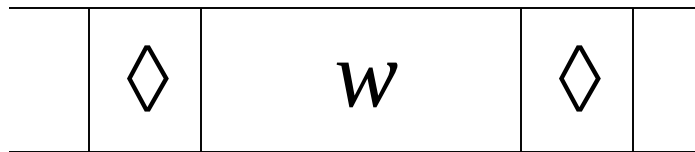
We prefer **unary** representation:

easier to manipulate with Turing machines

Definition:

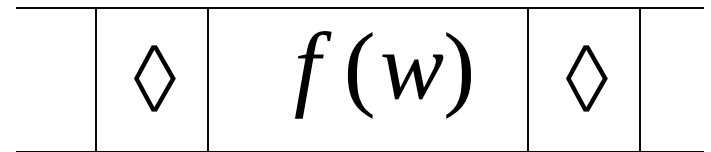
A function f is computable if
there is a Turing Machine M such that:

Initial configuration



q_0 initial state

Final configuration



q_f final state

For all $w \in D$ Domain

In other words:

A function f is computable if
there is a Turing Machine M such that:

$$q_0 w \xrightarrow{*} q_f f(w)$$

Initial

Configuration

Final

Configuration

For all $w \in D$ Domain

Example

The function $f(x, y) = x + y$ is computable

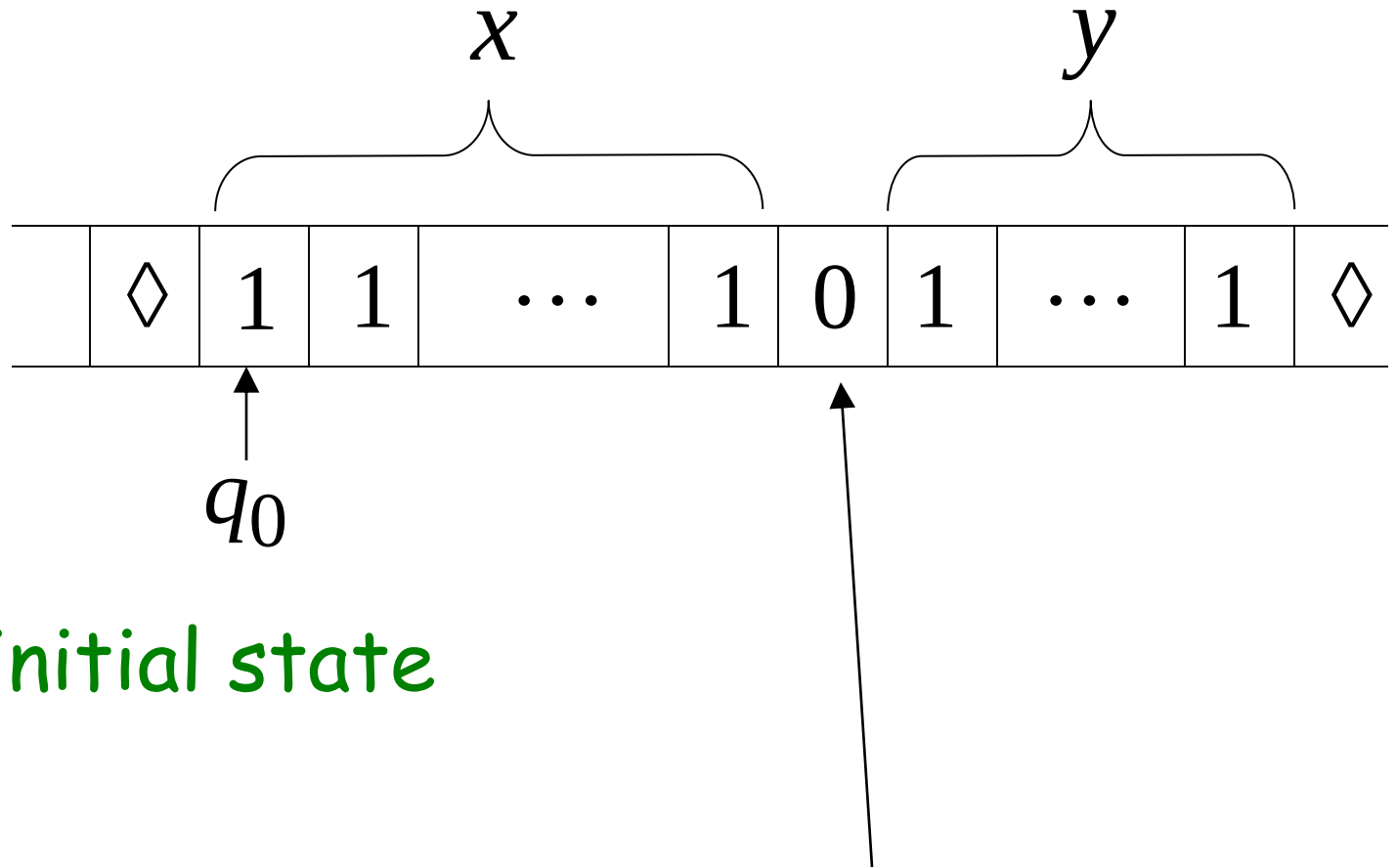
x, y are integers

Turing Machine:

Input string: $x0y$ unary

Output string: $xy0$ unary

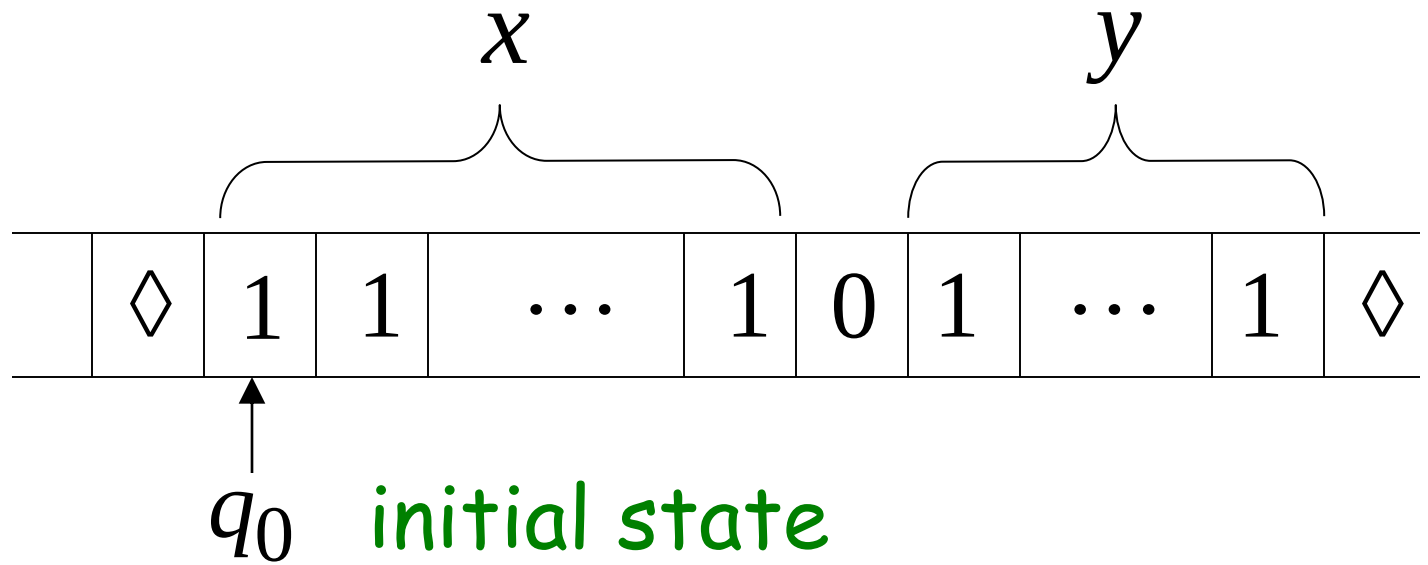
Start



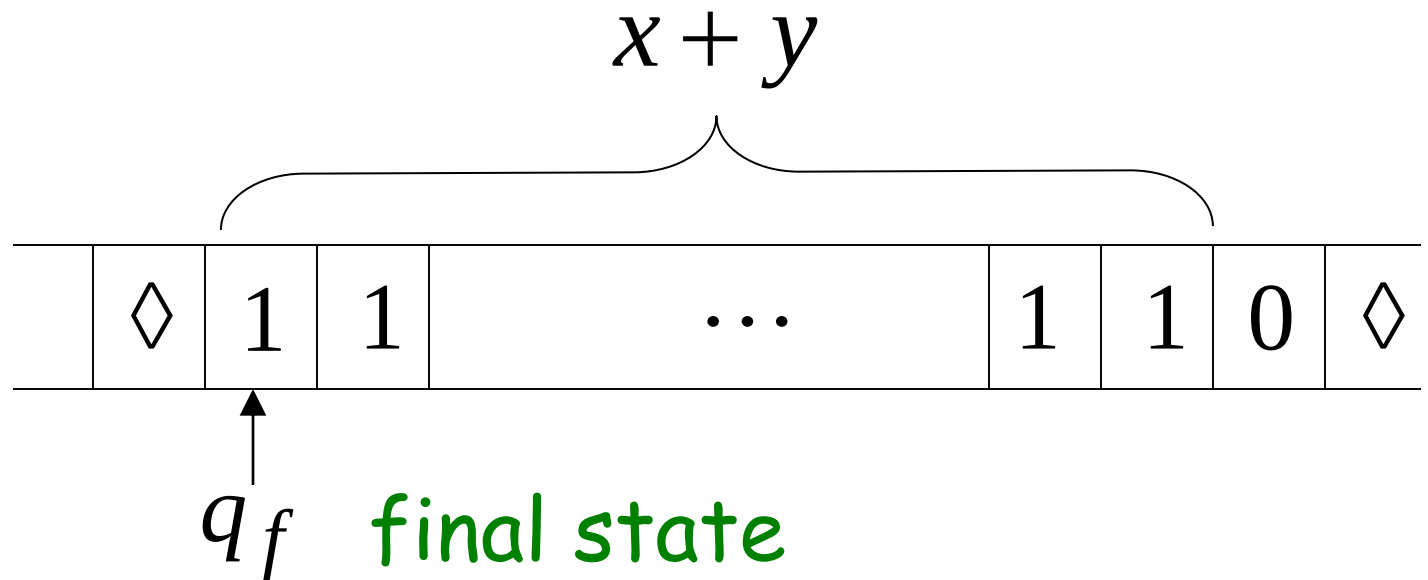
initial state

The 0 is the delimiter that separates the two numbers

Start

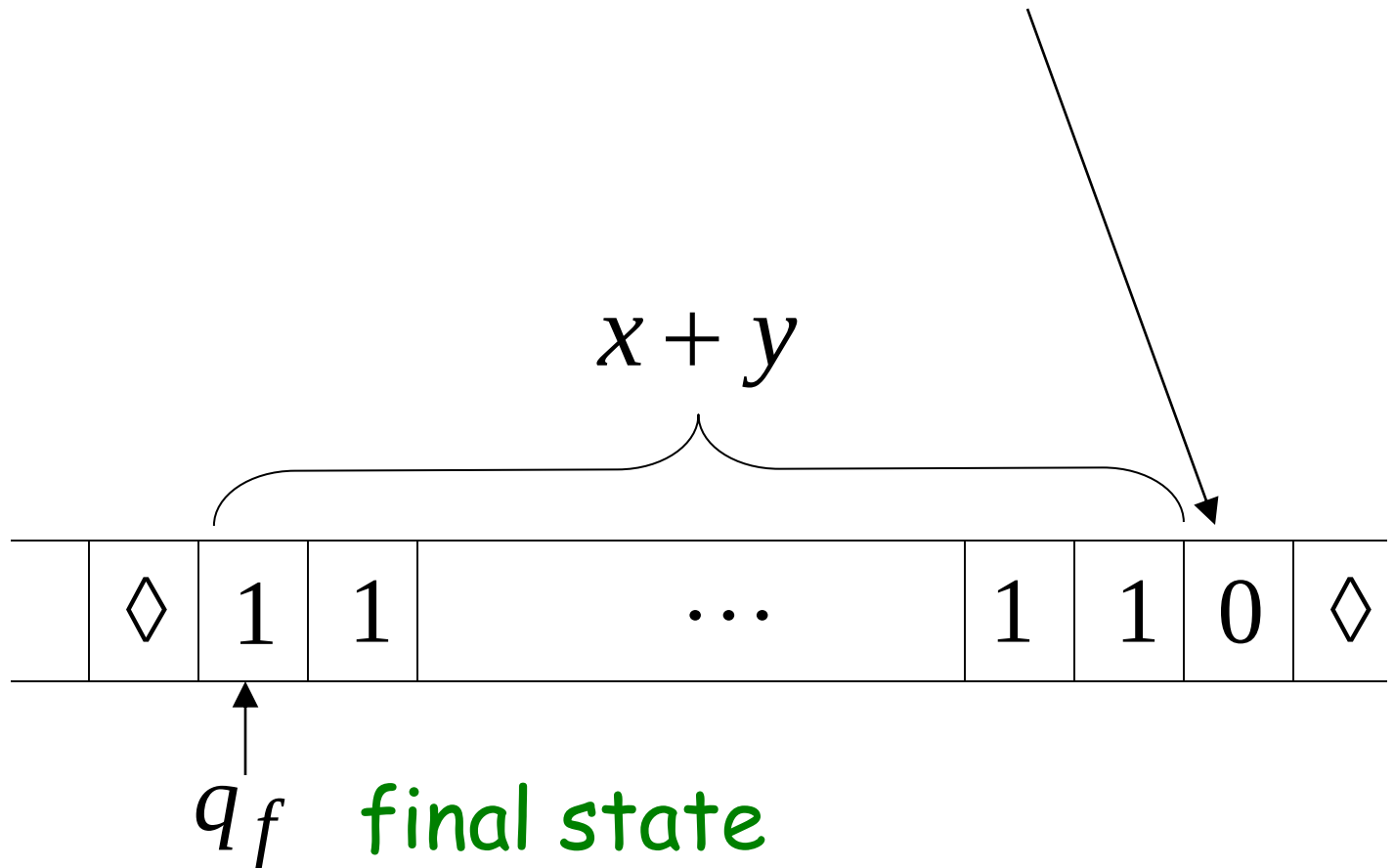


Finish

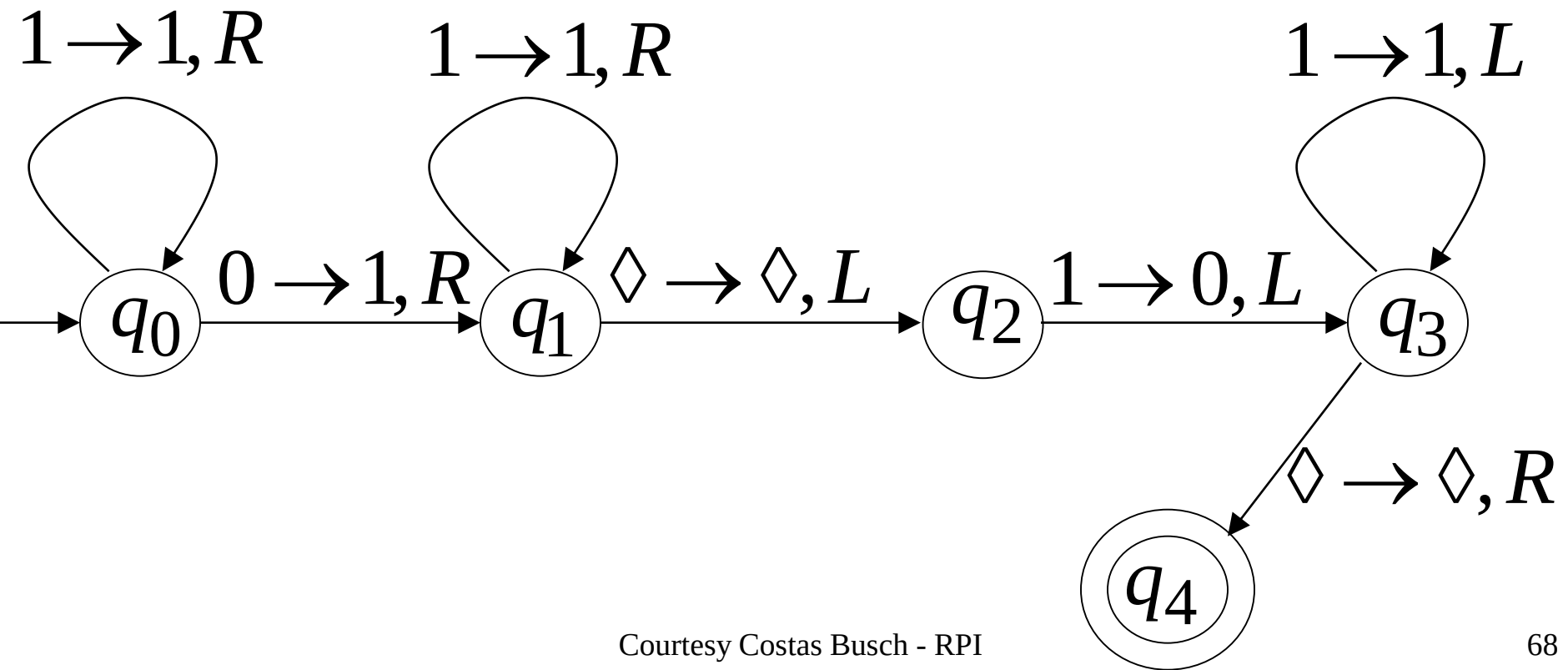


The 0 helps when we use
the result for other operations

Finish



Turing machine for function $f(x, y) = x + y$

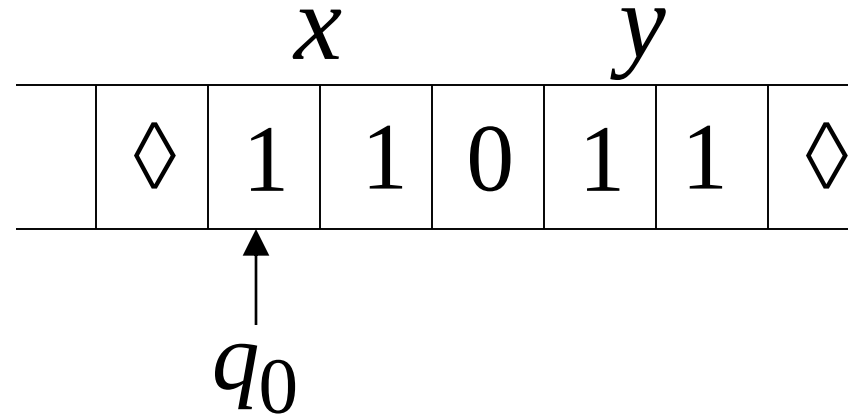


Execution Example:

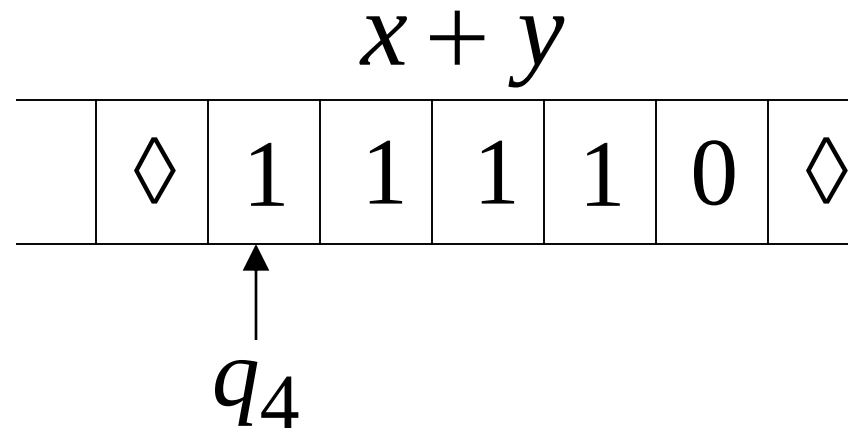
$x = 11$ (2)

$y = 11$ (2)

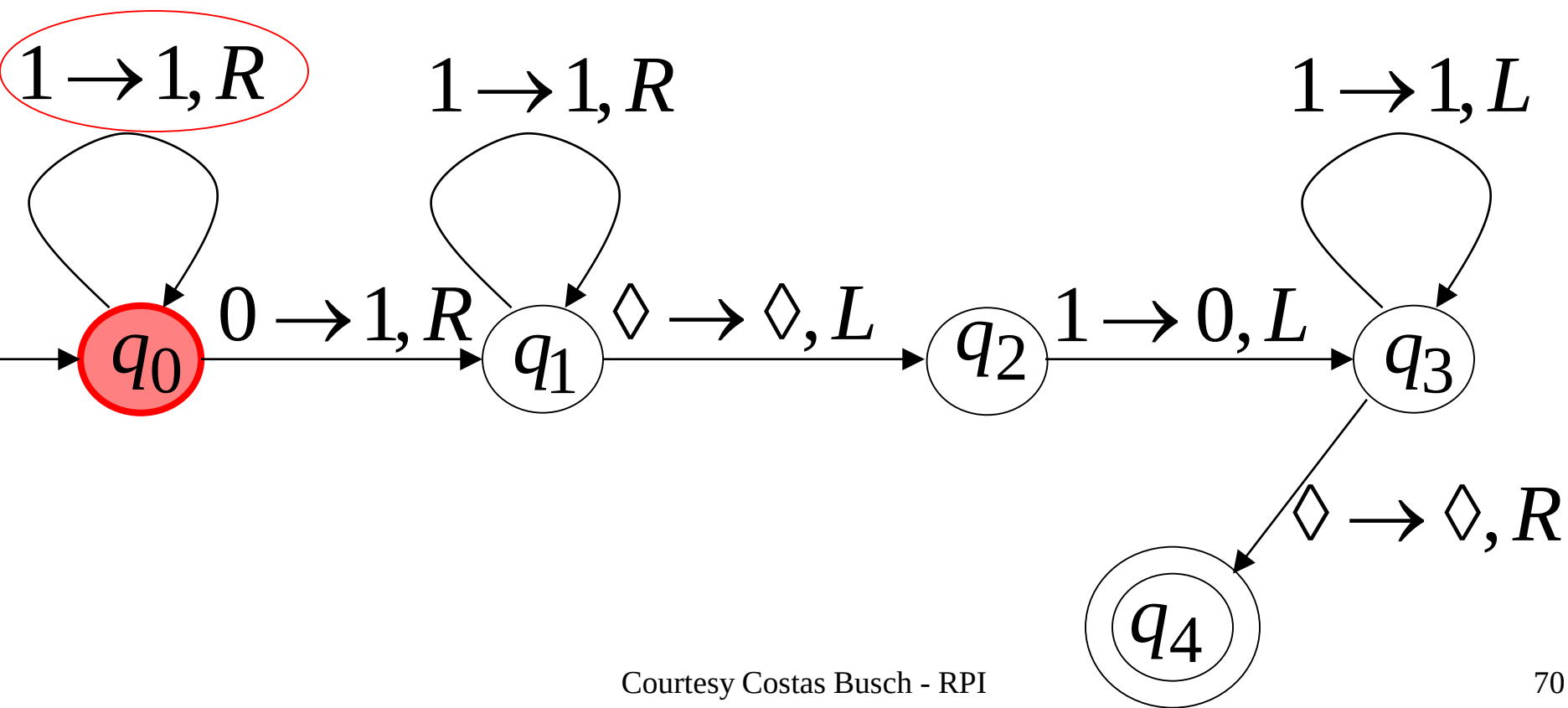
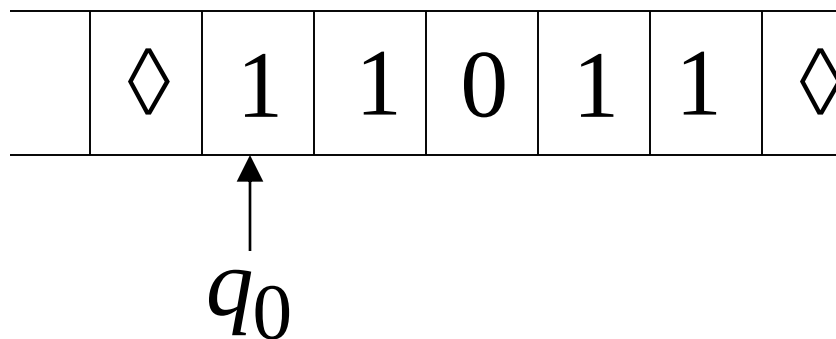
Time 0



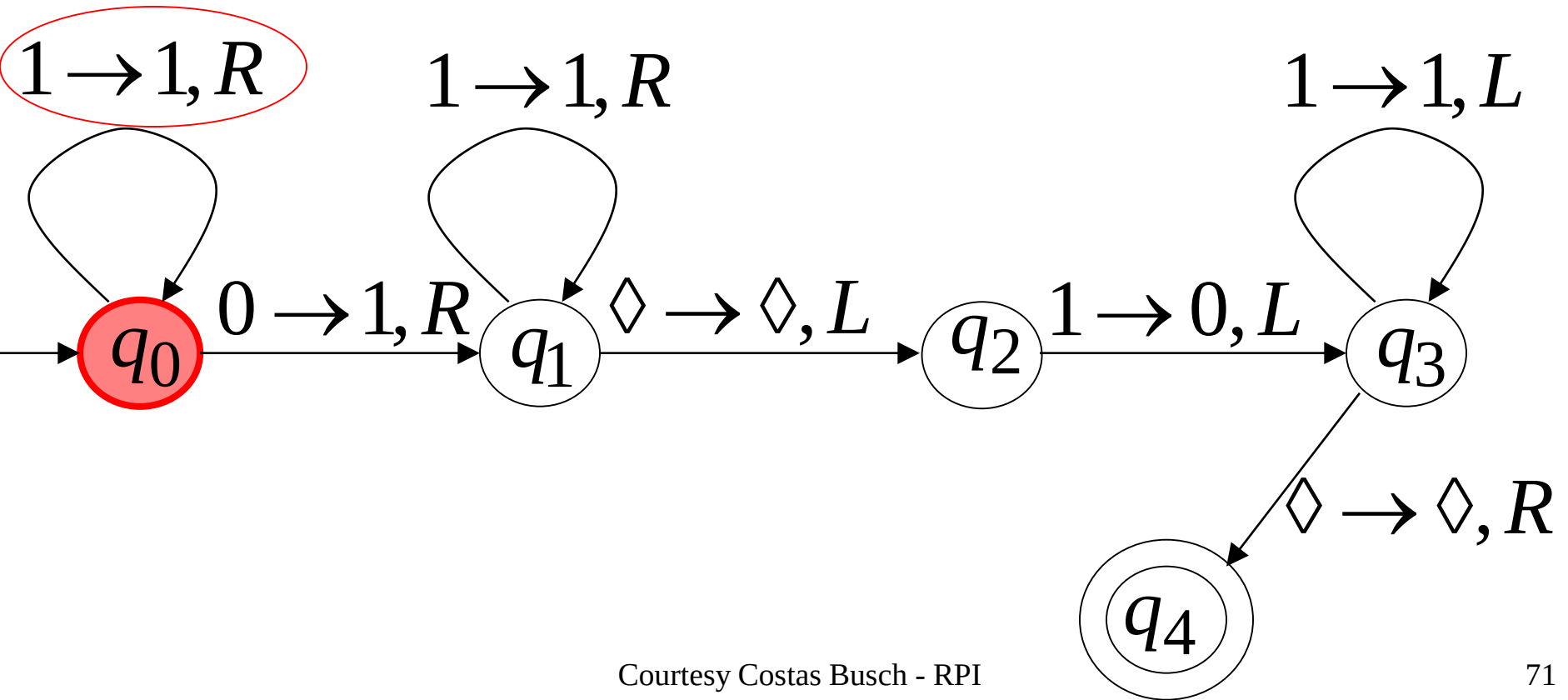
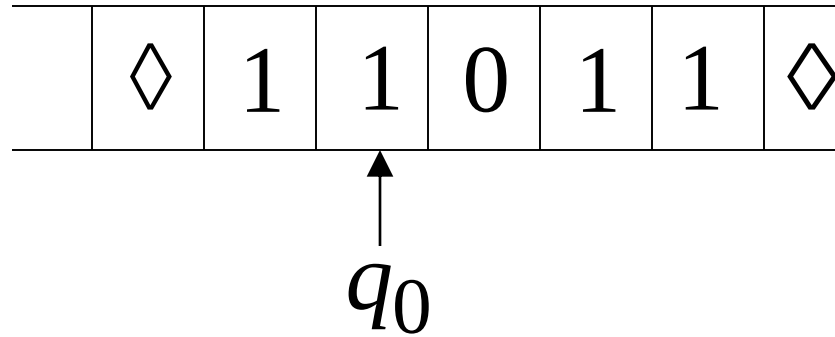
Final Result



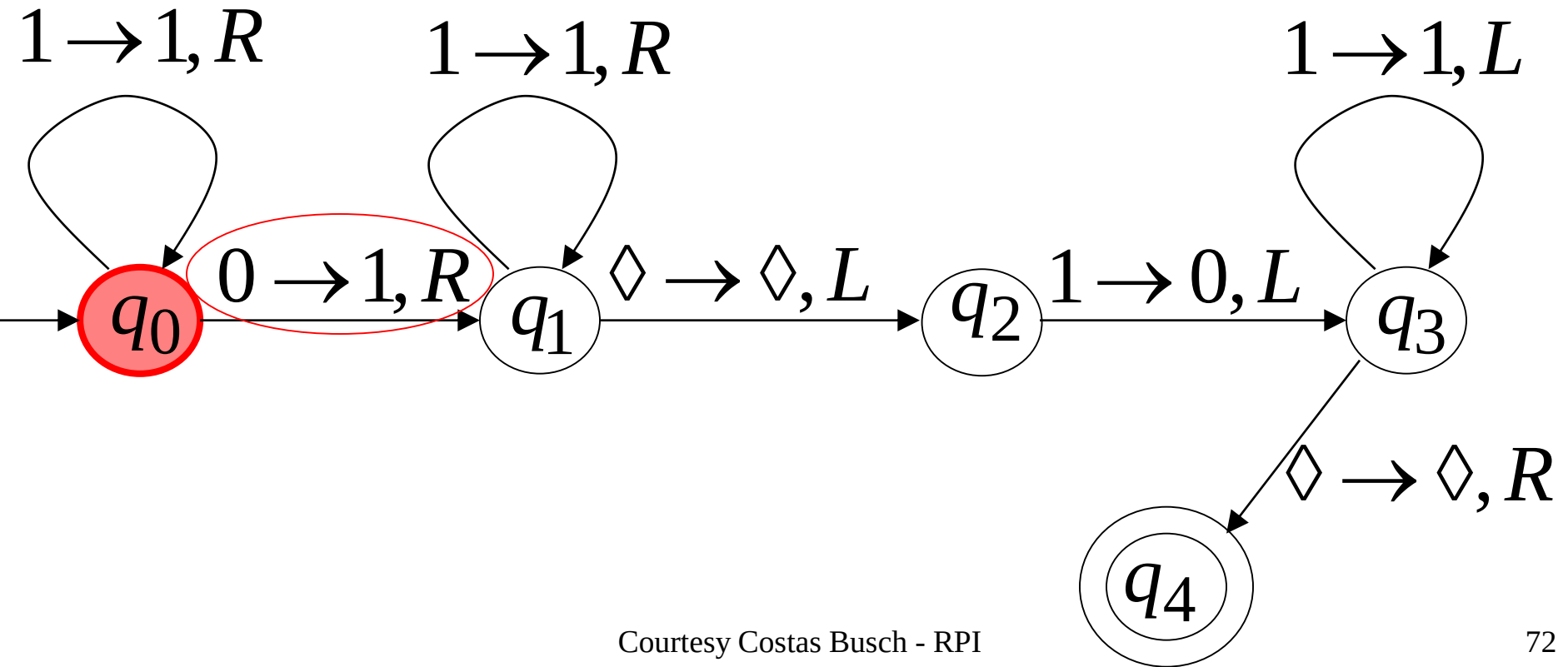
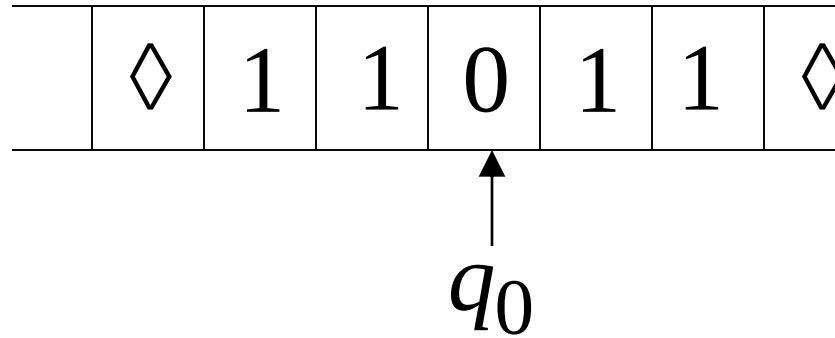
Time 0



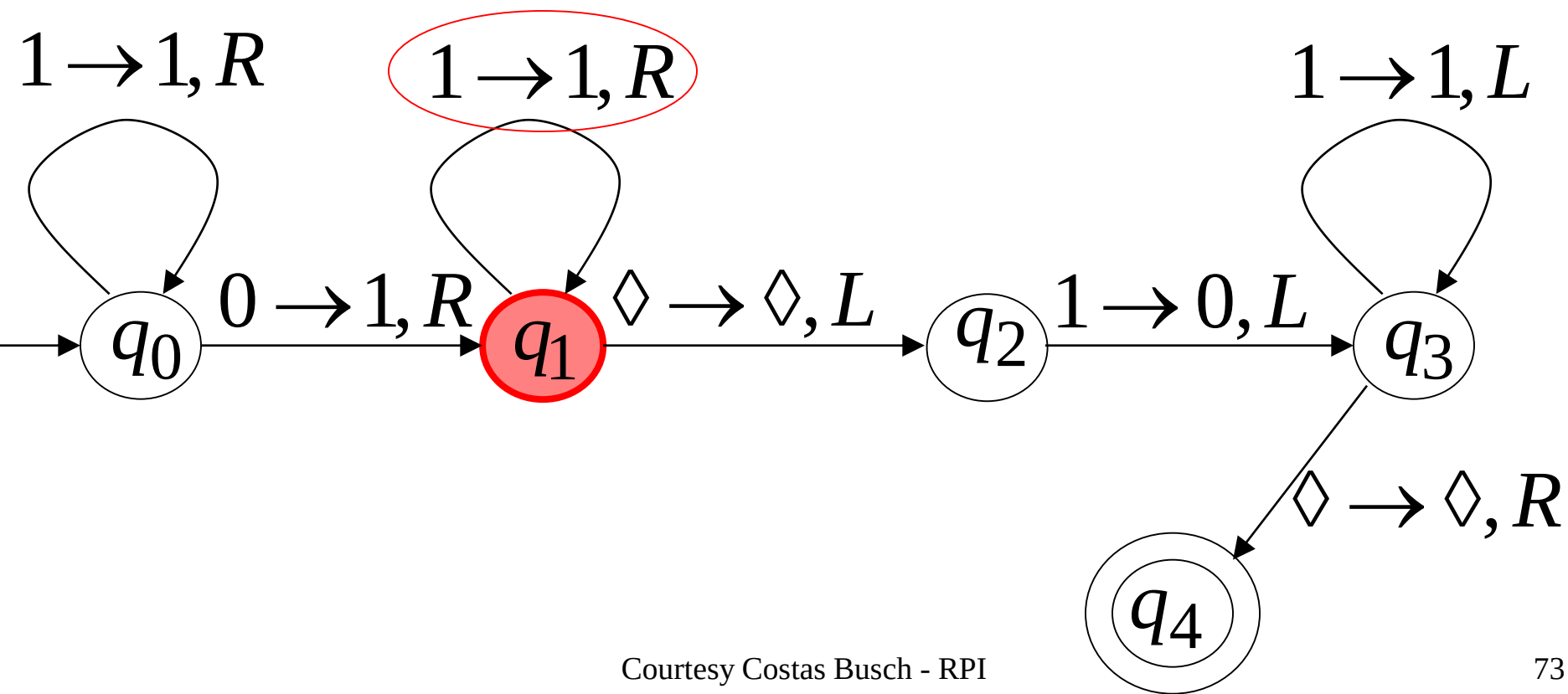
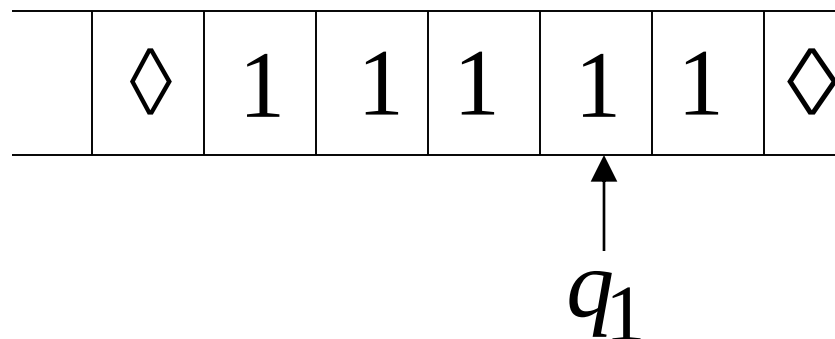
Time 1



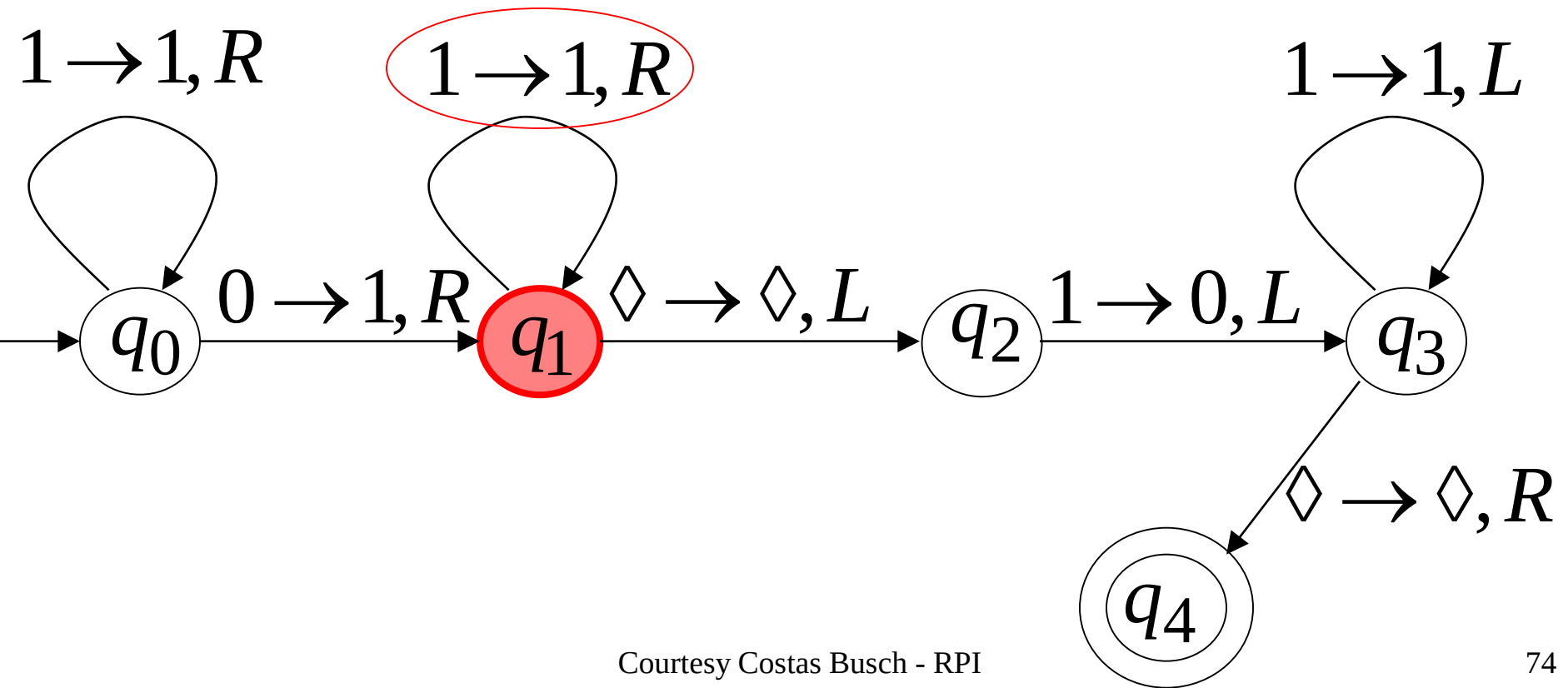
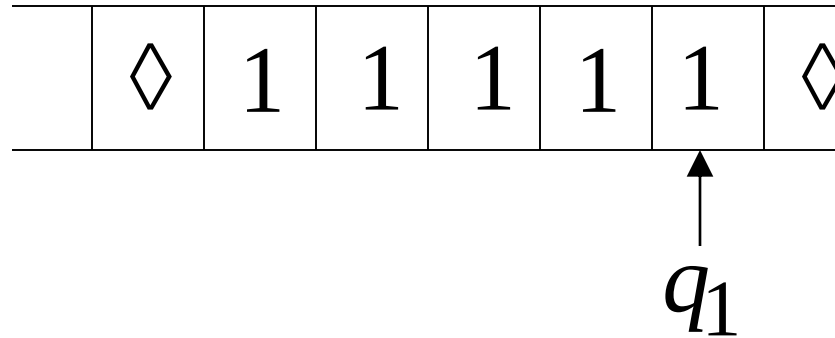
Time 2



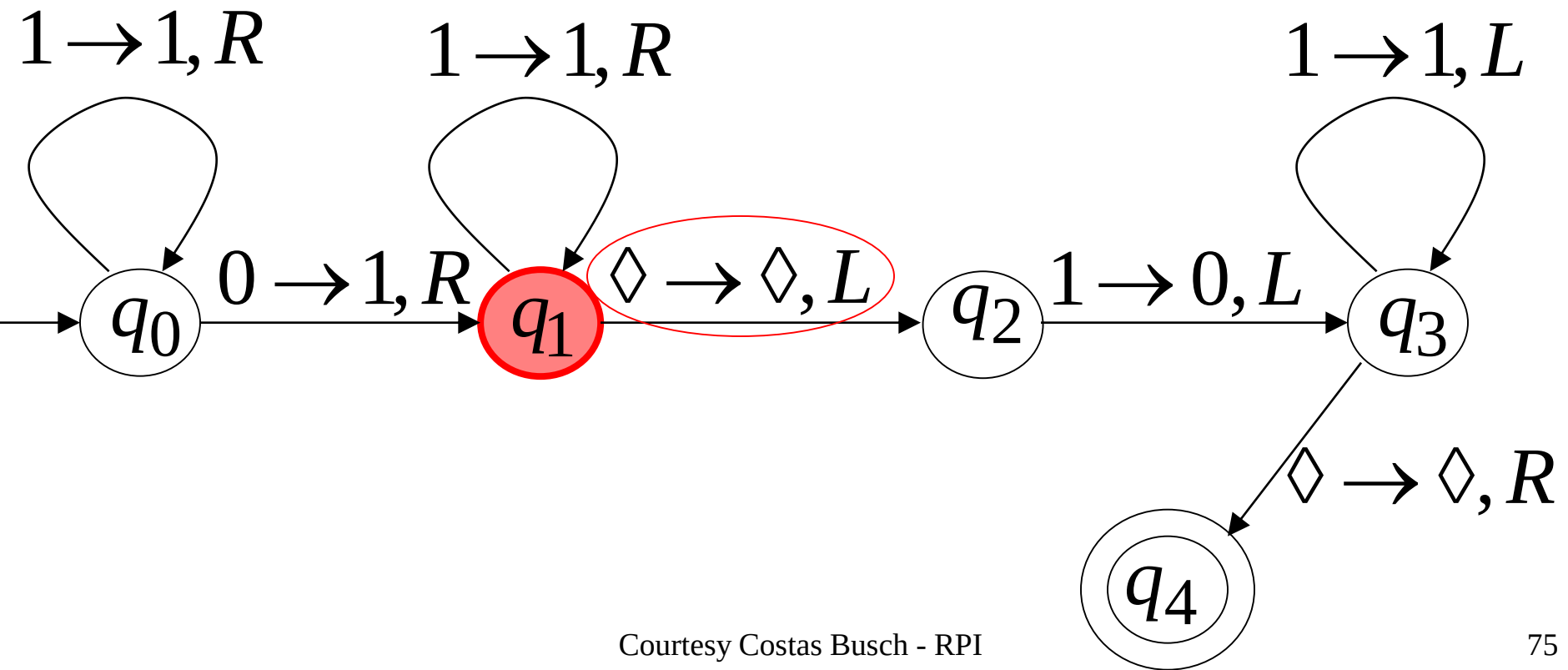
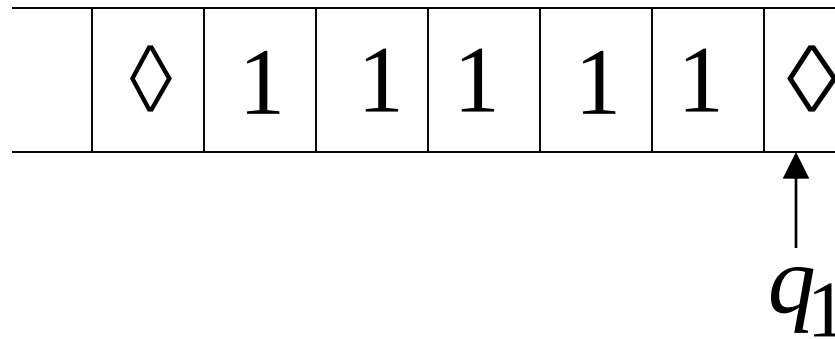
Time 3



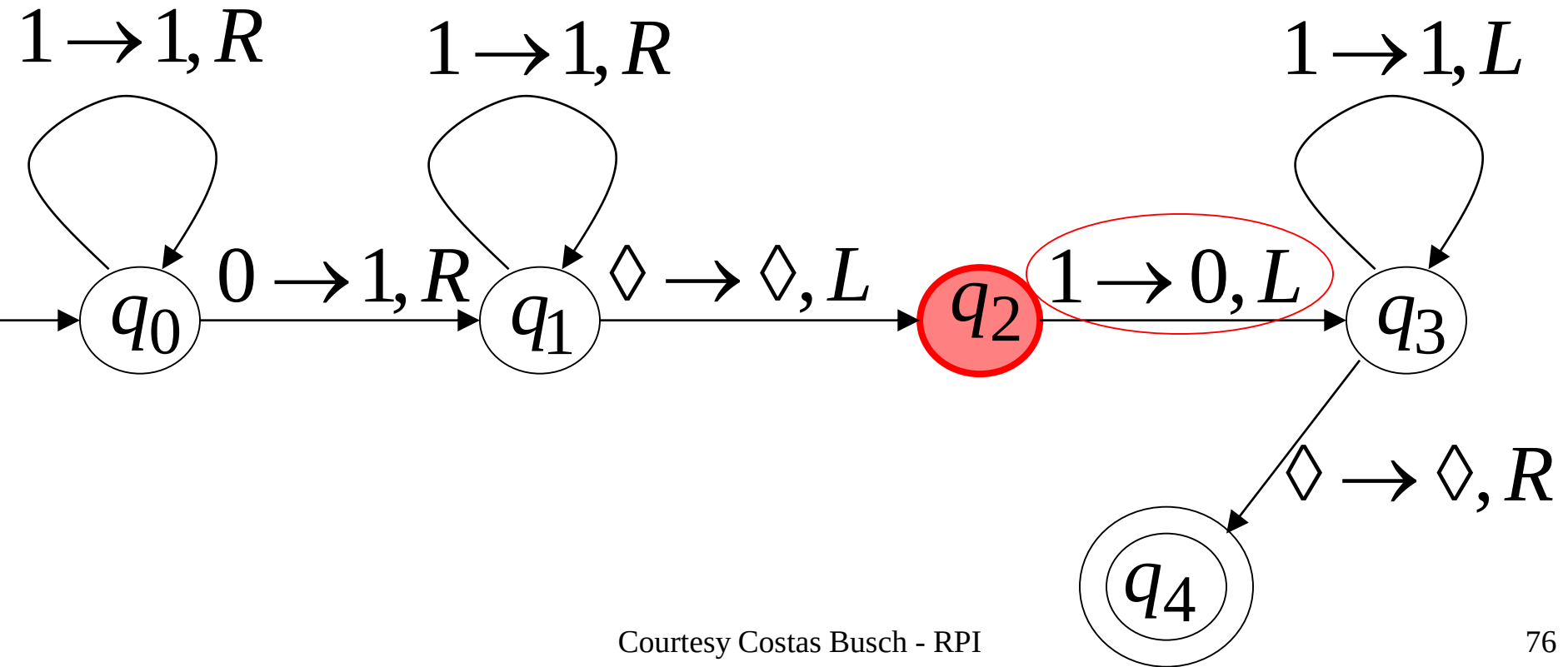
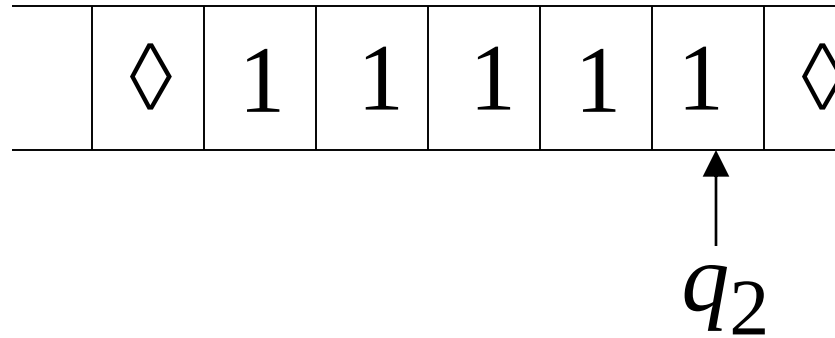
Time 4



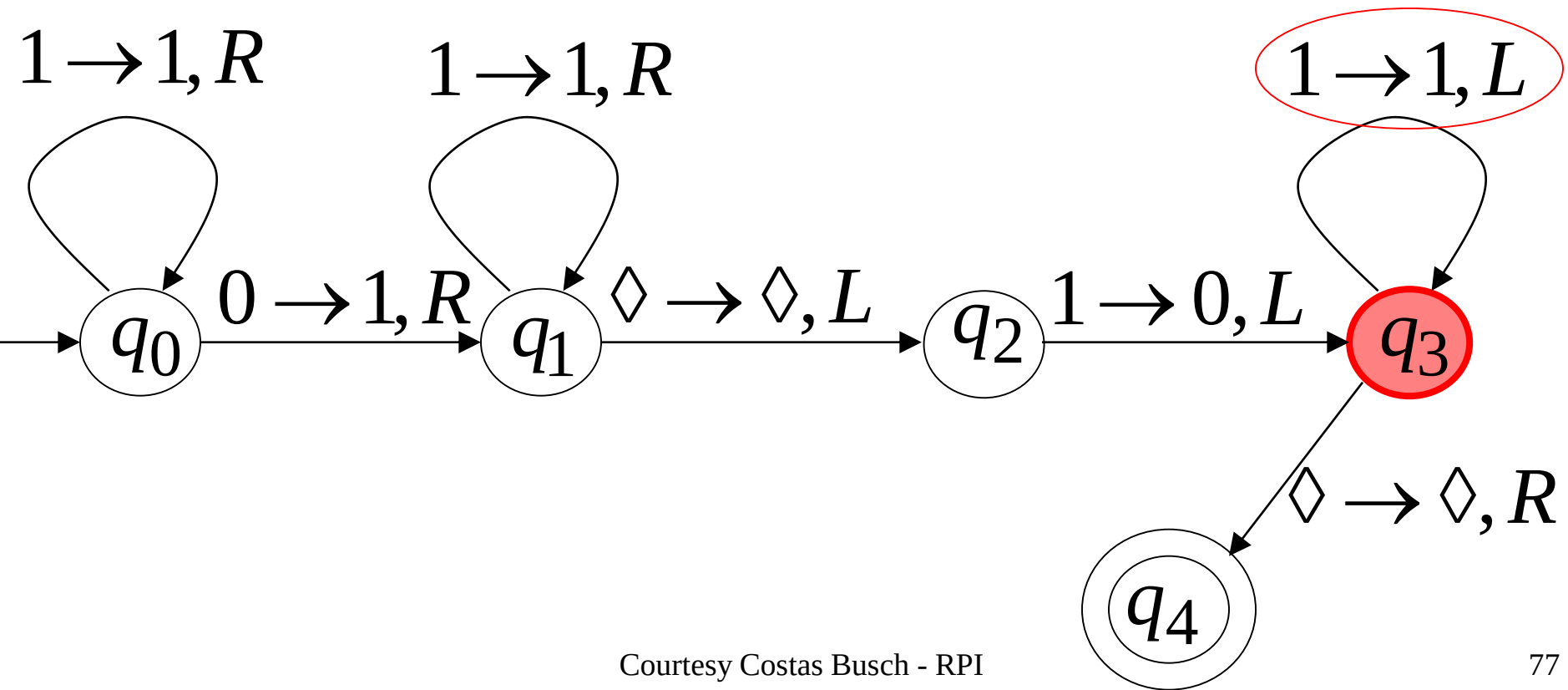
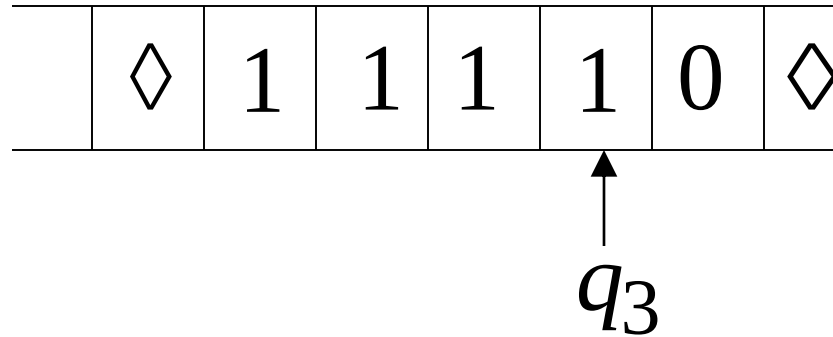
Time 5



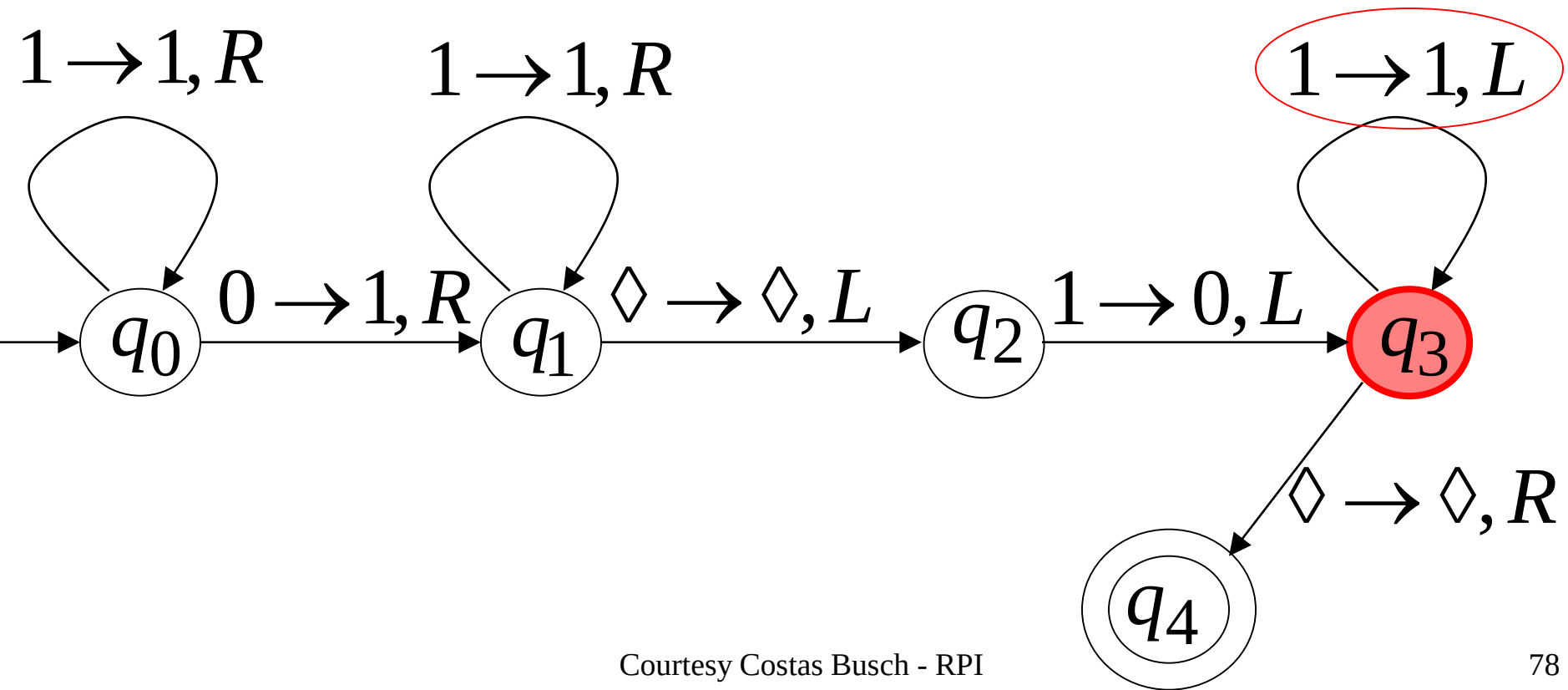
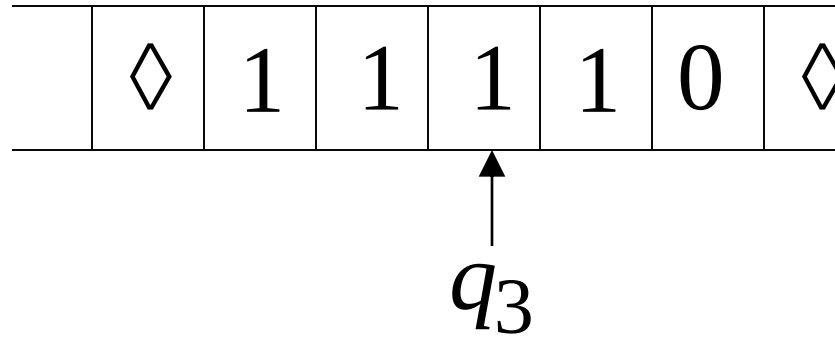
Time 6



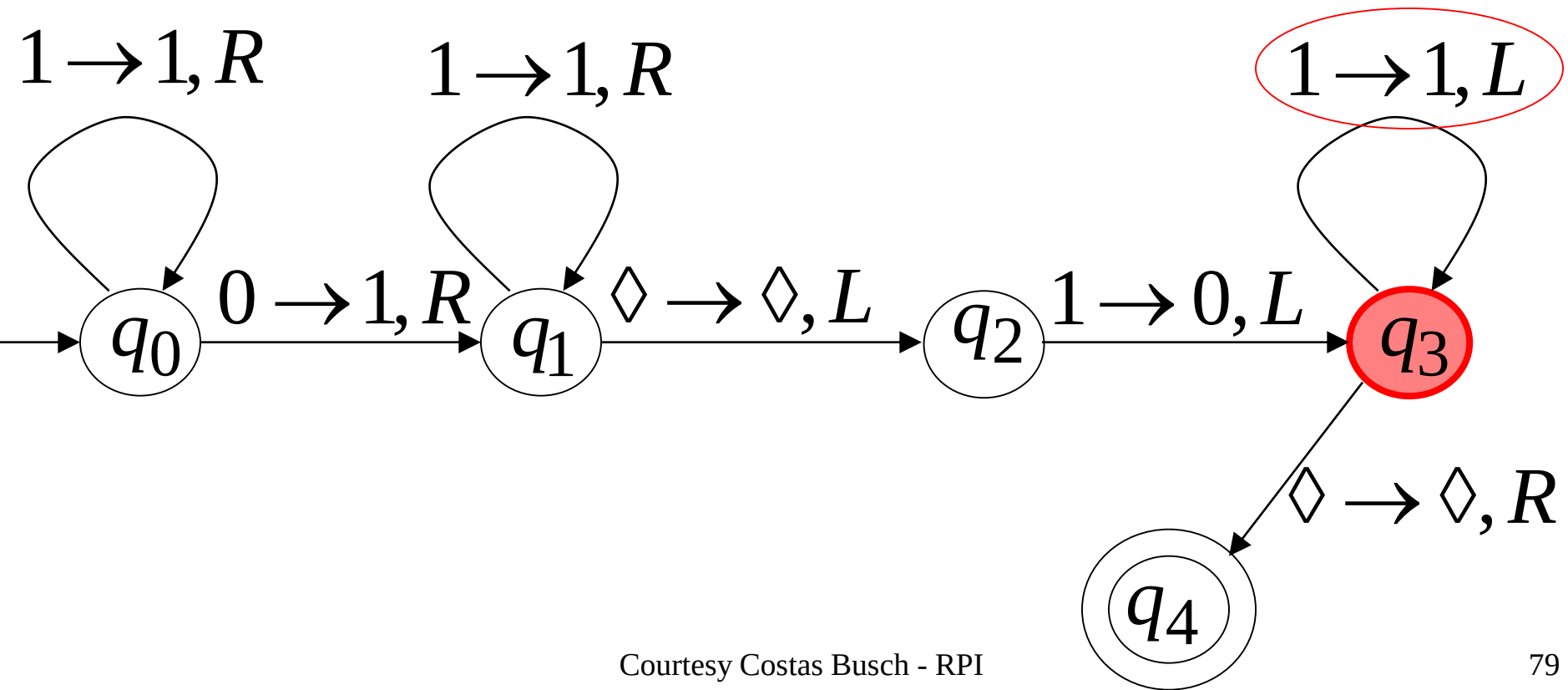
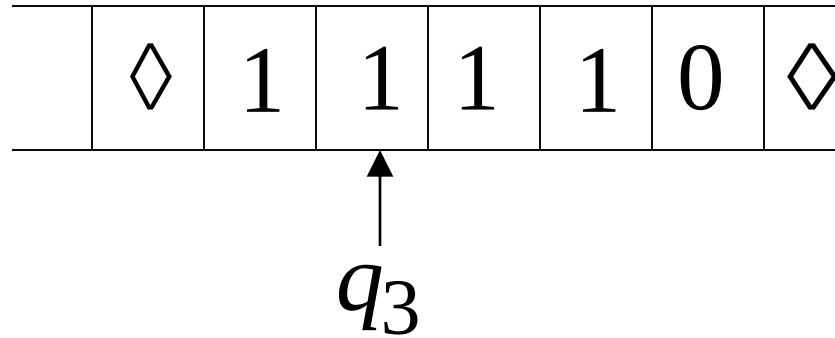
Time 7



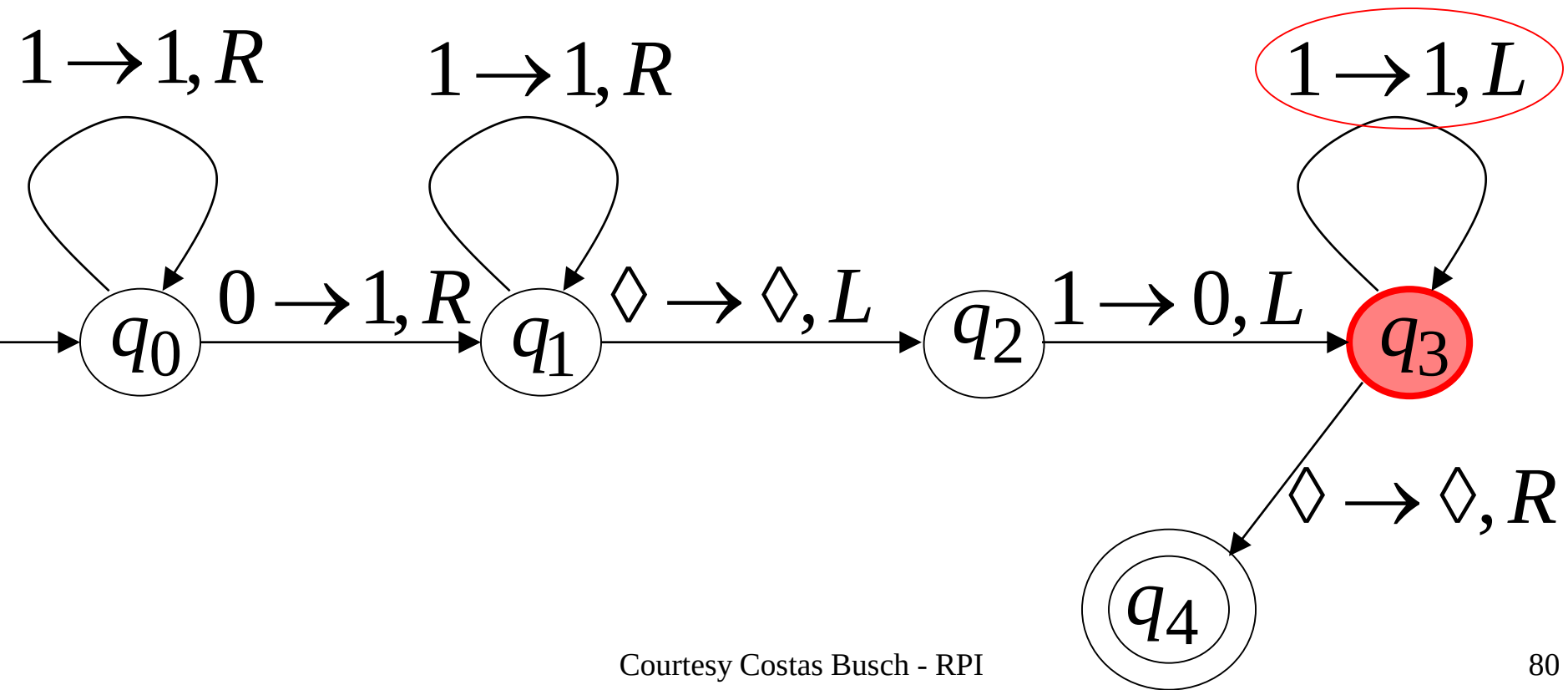
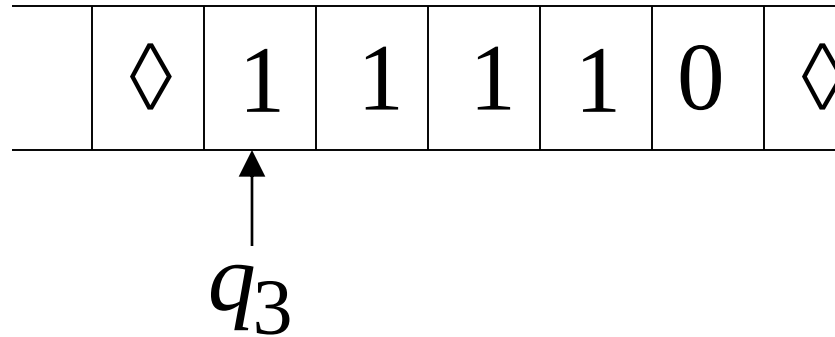
Time 8



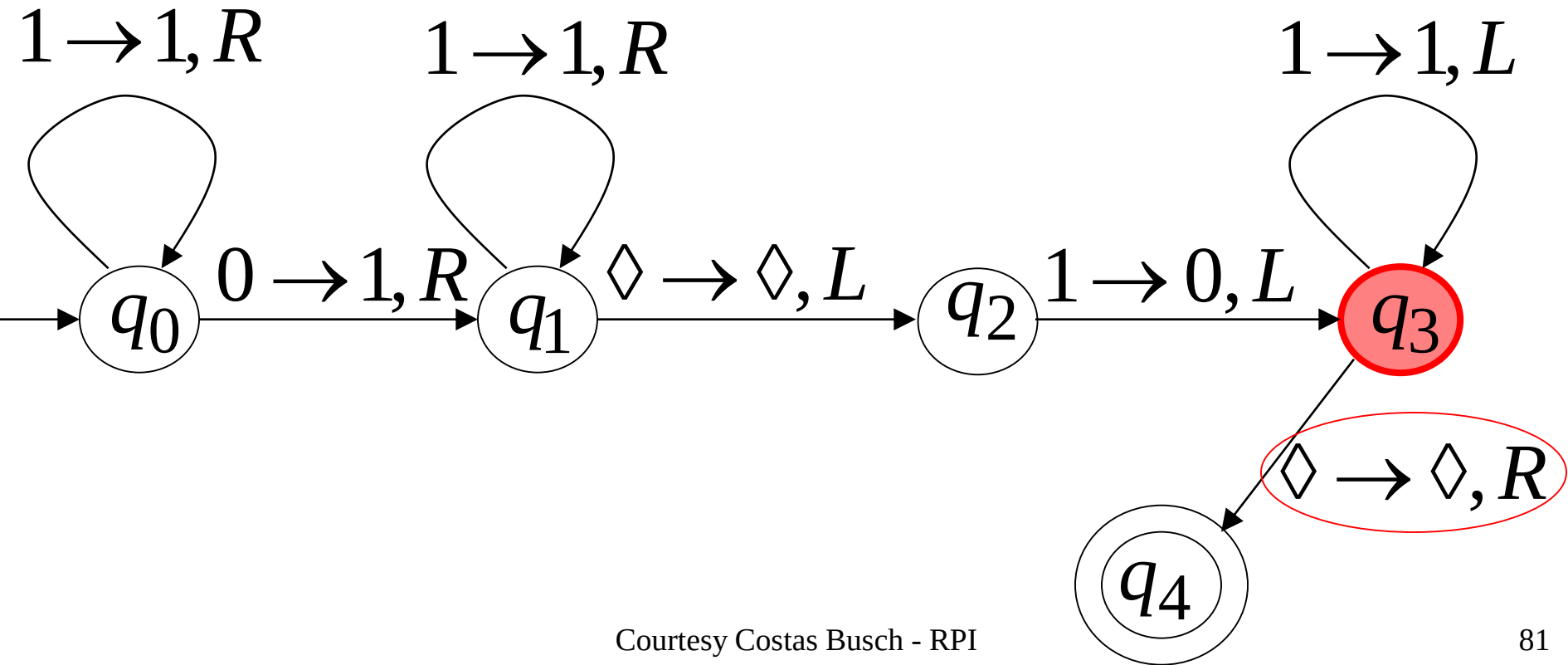
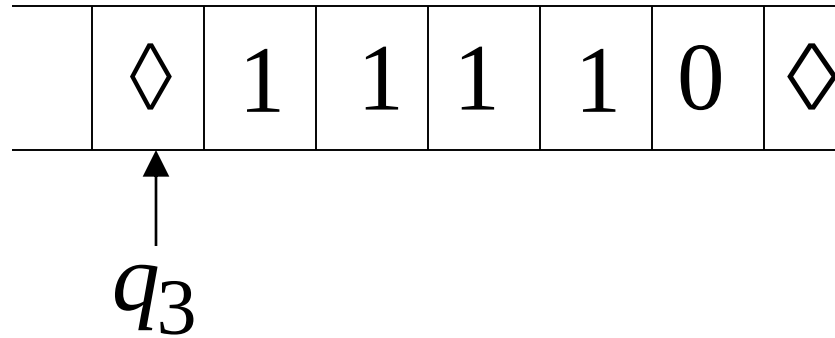
Time 9



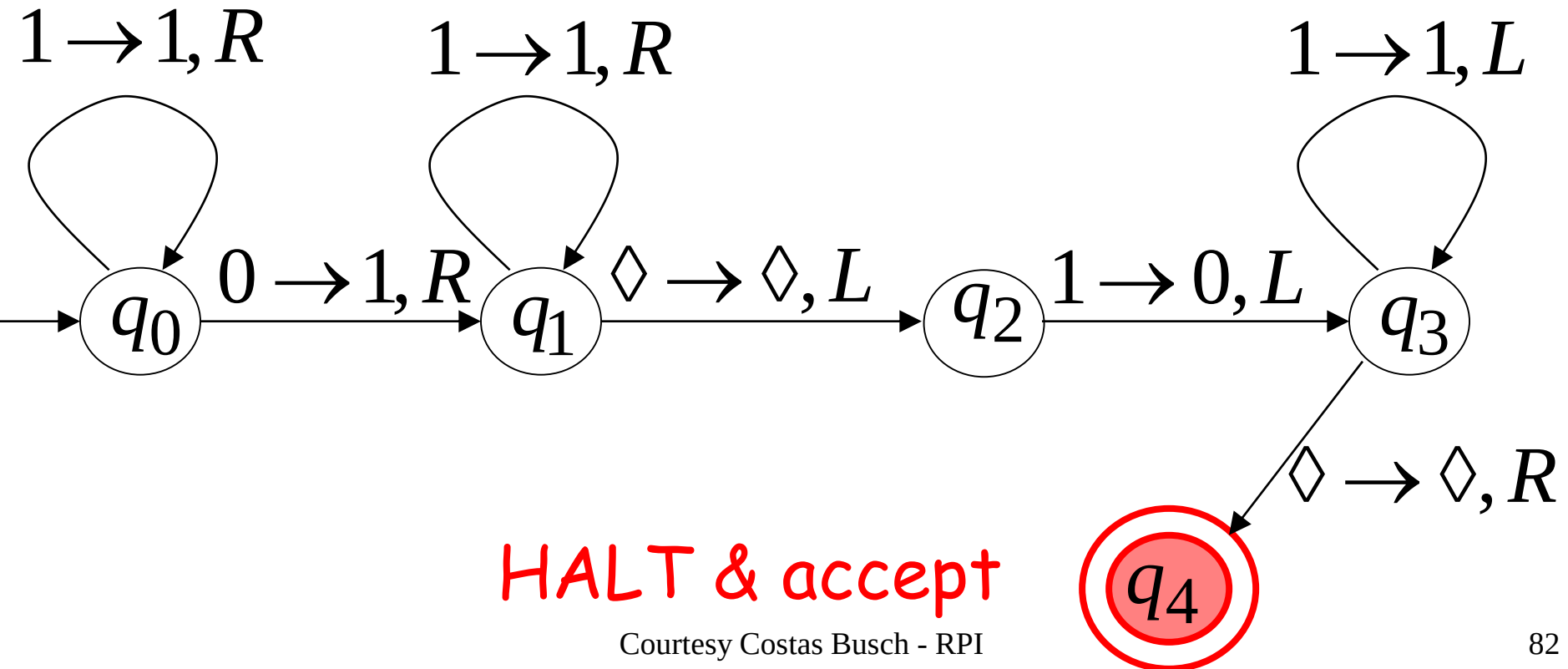
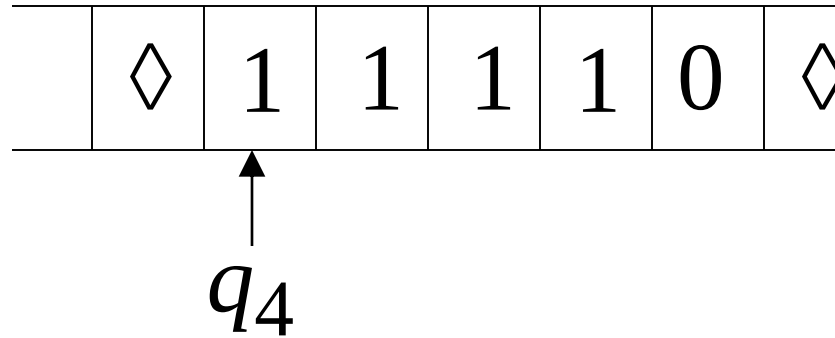
Time 10



Time 11



Time 12



Another Example

The function $f(x) = 2x$ is computable

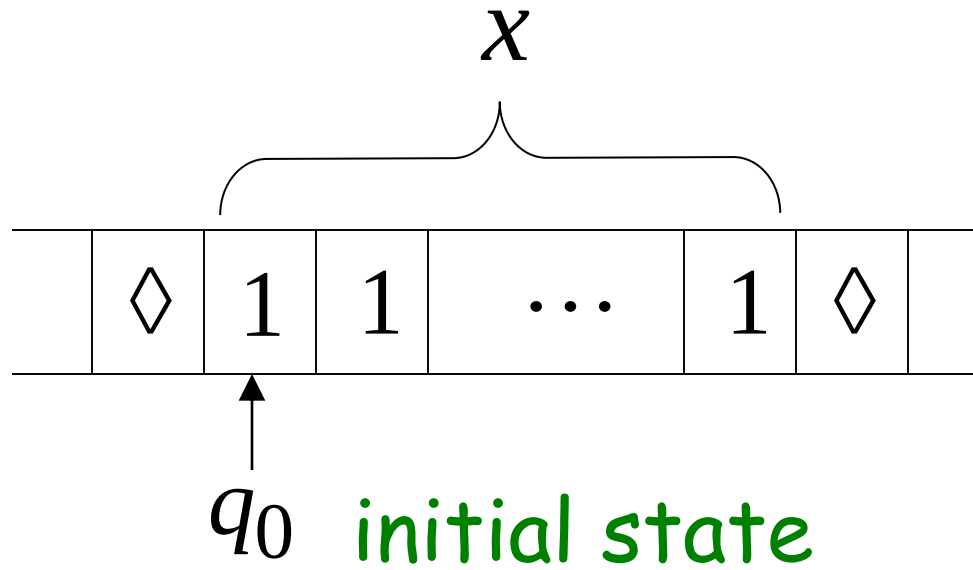
x is integer

Turing Machine:

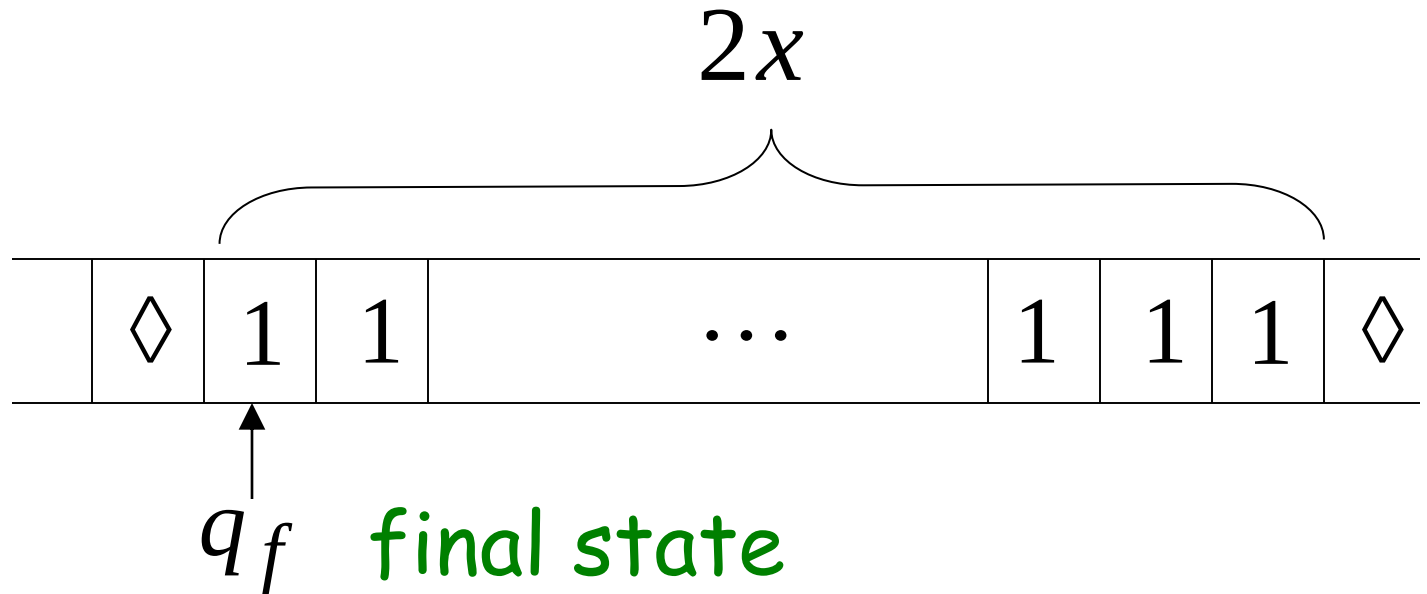
Input string: x unary

Output string: xx unary

Start



Finish

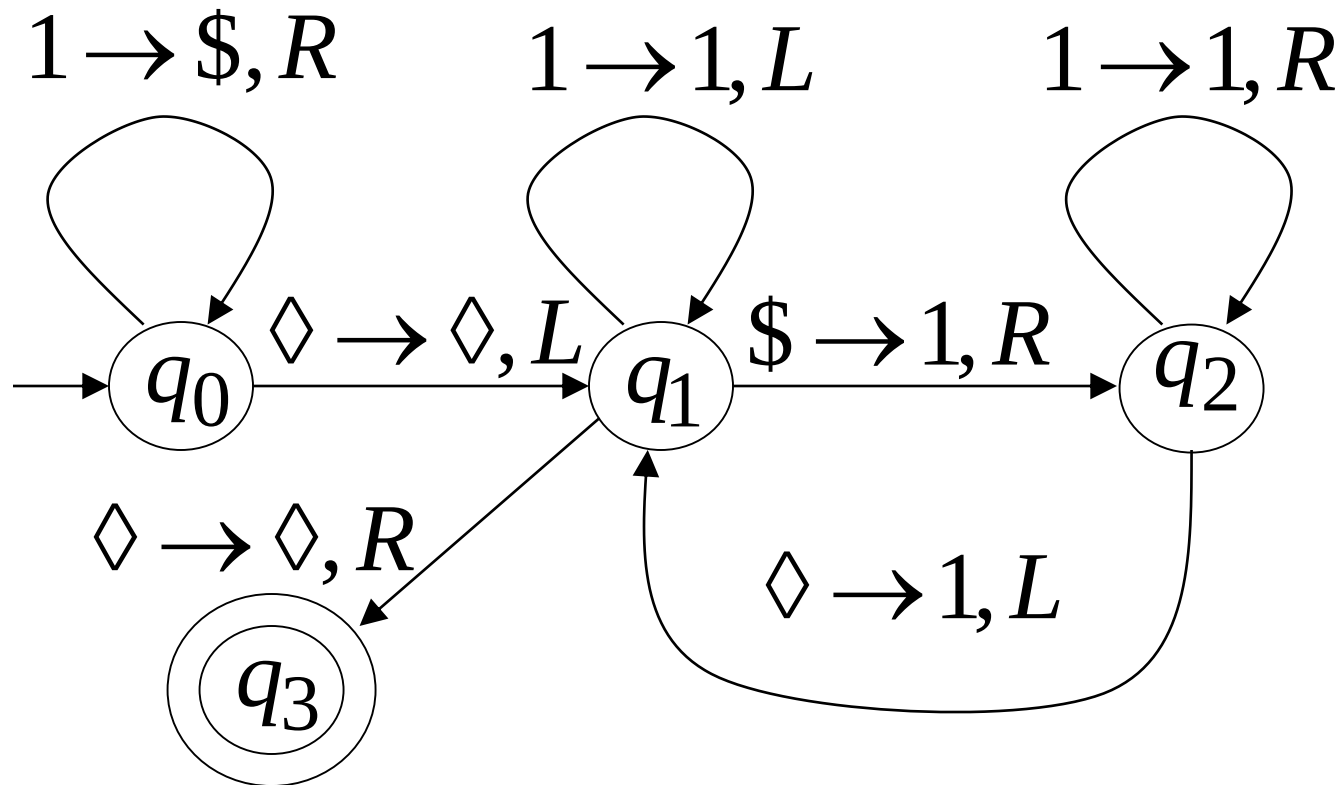


Turing Machine Pseudocode for $f(x) = 2x$

- Replace every 1 with \$
- Repeat:
 - Find rightmost \$, replace it with 1
 - Go to right end, insert 1

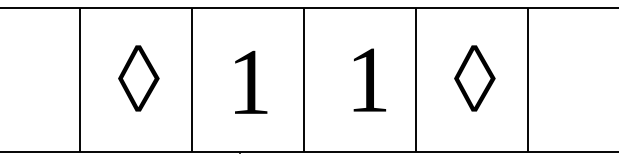
Until no more \$ remain

Turing Machine for $f(x) = 2x$



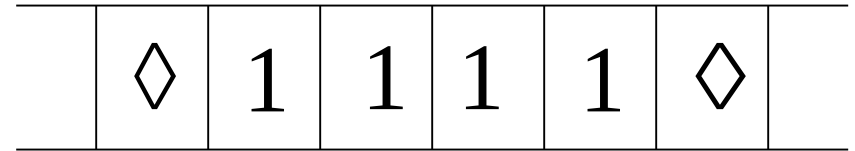
Example

Start



q_0

Finish



q_3

